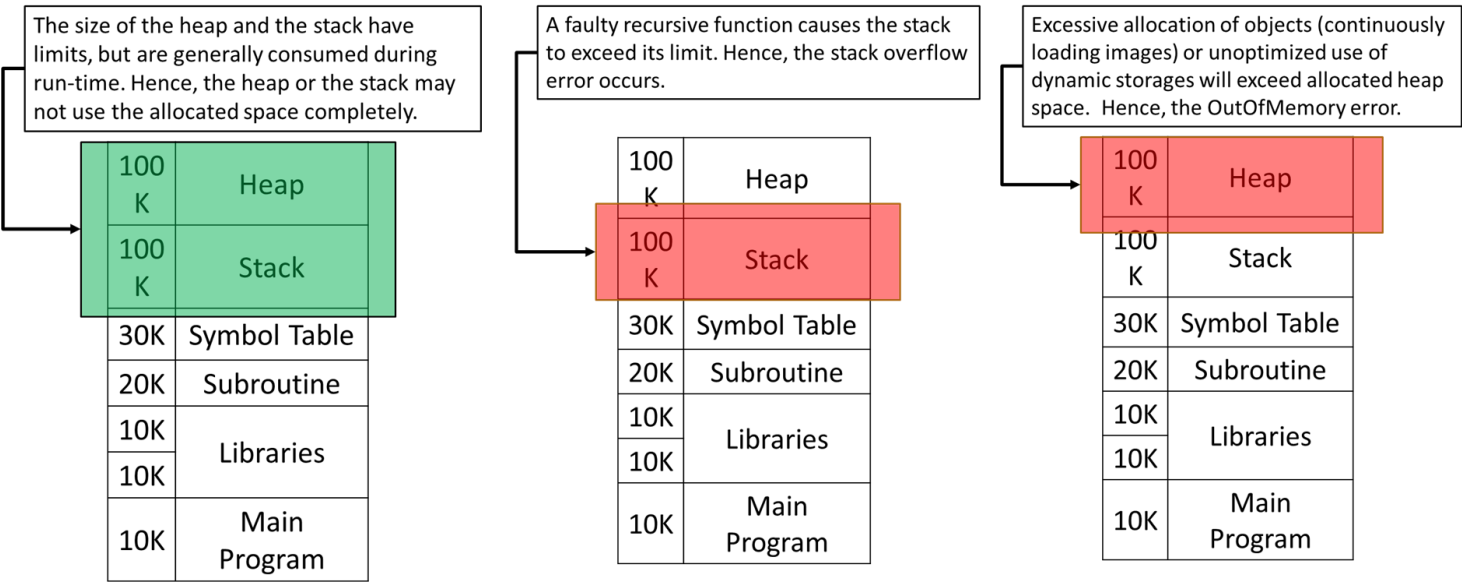
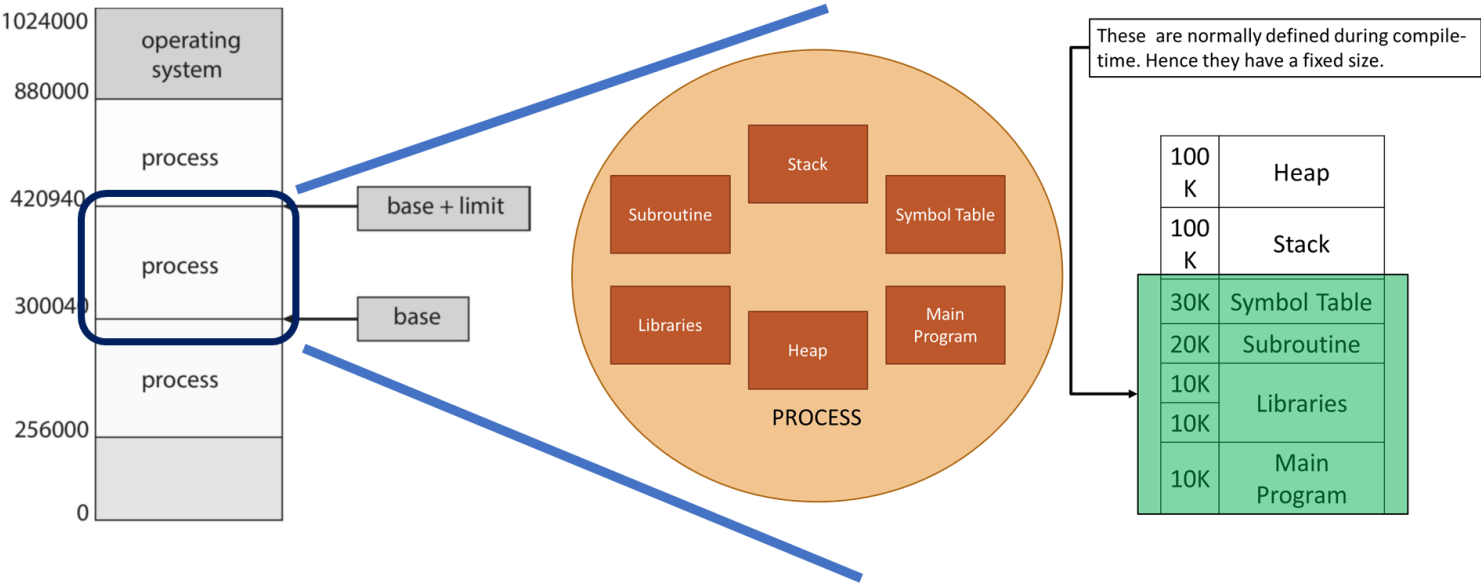


- Objective
- How OS manages its main memory for multiple processes.
 - How to emulate memory management for a multi-tasking OS.

- Overview
- There are several kinds of memory management models. We will only discuss:
 - Contiguous memory or flat memory model
 - Paging
 - Demand paging

- Basic Principles
- Most OS pre-allocate memory for each process, depending on its relative size and computational demand. The pre-allocated memory can be w ritten/freed when executing instructions → malloc(), dealloc(), pointers, etc.
 - Each process have a separate memory space.
 - Start address and end address present in process info.
 - OS also has a share of its memory space!
 - Processes cannot execute without their resources loaded into memory.



An Illustration of Flat Memory Allocation

Dynamic Memory Allocation Problem

- At the very start of the OS program, memory are divided into N partitions, with pre-defined sizes.

10K	
10K	
10K	
10K	
10K	
10K	
10K	
10K	
--	OS



Dynamic Memory Allocation Problem

- Before the process executes in the CPU, the OS allocates a hole big enough for the process.

10K	
10K	
10K	
10K	
10K	
10K	
10K	
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Processes that require larger memory should need more holes.
- Consider P2 that requires 20K memory

10K	
10K	
10K	
10K	
10K	
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Processes sometimes require memory that is not divisible by partition size
- Consider P3 that requires 14K memory.
- P3 still needs to consume an additional 10K hole!

10K	
10K	
10K	
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Unused memory in hole results in **internal fragmentation**.

P3 needs additional 10K hole. But 4K only used. Hence, internal fragmentation of 6K.

10K	
10K	
10K	
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- QUESTION:** What will the OS do if a process arrives, but the memory is full? (+1)

Process 5 ❌

10K	Process 4
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- Incoming processes to be scheduled in CPU may be delayed if memory space is full.
- OS may do the following:
 - Wait until a process finishes execution, to free up memory space
 - Perform swapping. A process in the waiting queue will have to release its memory and transfer it back to a **backing store**.

Process 5 ❌

10K	Process 4
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- If swapping is enabled, the swapped process, when re-scheduled back to the CPU, reloads its resources back to memory from the backing store.
- Both options have tradeoff costs!
 - Waiting until a process finishes – lower CPU throughput.
 - Swapping – moving back the process to memory incurs additional CPU cost.

10K	Process 4
10K	Process 3
10K	Process 2
10K	Process 1
--	OS



Dynamic Memory Allocation Problem

- When a process finishes execution, it can immediately free memory for a future process.

10K	Process 4
10K	Process 3
10K	Process 2
10K	
--	OS



Dynamic Memory Allocation Problem

- Suppose a new process arrives that demands 30K memory.
- We have enough memory space! But at different partitions/holes. What can the OS do?

Process 5

10K	Process 4
20K	
10K	Process 2
10K	
--	OS



Dynamic Memory Allocation Problem

- The OS cannot allocate P5 in memory! Enough memory space but fragmented!
- Partitions that are not **continuous** are called **external fragmentation**.
- In this example, external fragmentation is 30K. Meaning 30K space may be left unused because it has been divided.

10K	Process 4
10K	
10K	
20K	
10K	Process 2
10K	
10K	
--	OS

Dynamic Memory Allocation Problem

- QUESTION: Assume all processes have finished execution. What can we do to our memory space to lessen fragmentation? (+1)

30K	
20K	
20K	
10K	
--	OS

Dynamic Memory Allocation Problem

- OS may choose to do the following:
 - Revert back to its original partitioning scheme.
 - Maintain new partitions as is.
 - Continuous partitions may be merged, to avoid internal fragmentation.

30K	
20K	
20K	
10K	
--	OS

Dynamic Memory Allocation Problem

- Continuous partitions may be merged, to avoid internal fragmentation.
- P6 arrives that requires 32K memory.

30K	
20K	
20K	
10K	
--	OS

Dynamic Memory Allocation Problem

- Continuous partitions may be merged, to avoid internal fragmentation.
- P6 arrives that requires 32K memory.

48K	
32K	Process 6
--	OS

Swapping and the Backing Store

The diagram shows the interaction between the operating system, user space, and main memory on one side, and the backing store on the other. In the main memory, there are blocks for 'operating system', 'user space', and 'main memory'. In the backing store, there are blocks for 'process P1' and 'process P2'. An arrow labeled '1 swap out' points from the user space to process P1 in the backing store. Another arrow labeled '2 swap in' points from process P2 in the backing store back to the user space.

General steps:

- Continuously allocate processes in the user space.
- If there are no free user space, select a process that could be put to a backing store. To put in backing store, save necessary information about the process. E.g. PID, program counter, certain allocations/variable declarations performed.
- Free up the block of the selected process
- Put the new process in the recently freed up block.

Handwritten notes:

- ↓ min = fragmentation
- ↑ CPU utilization
- X processes actively executing in CPU
- 1. backing store
- 2. loading it again
- core 1
- context-switching

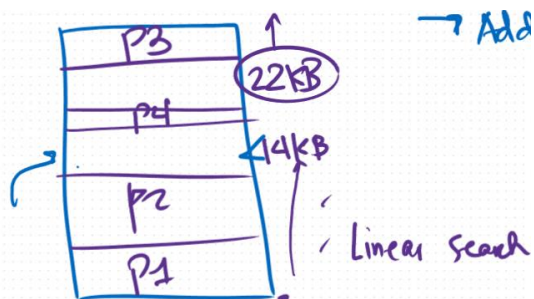
- Swapping** is a memory management technique used by operating systems to temporarily move a process or a portion of a process from main memory (RAM) to secondary storage (usually a disk) and vice versa. The secondary storage is the **backing store**.
- The primary goal of sw apping is to free up space in the main memory for active processes. When a process is swapped out, its contents are moved to the secondary storage, allowing other processes to utilize the freed-up memory.
- Process:** When a process is swapped out, all of its contents are transferred to a reserved space on the disk (swap space). The operating system maintains a swapping policy to determine when to swap processes in and out based on priority, demand, or other criteria.

- **Performance Impact:** While swapping can prevent memory exhaustion and allow more processes to run, excessive swapping can lead to performance degradation due to increased I/O operations between main memory and secondary storage.

The flat memory model follows a combination of **first-fit** and **quick-fit** approaches. Here are the primary fit approaches:

1. First Fit:

- **Description:** In the first-fit approach, the operating system allocates the first available block of memory that is large enough to accommodate the process's size.
- **Advantages:** Simple and easy to implement. Quick allocation of memory.
- **Disadvantages:** May result in fragmentation, both internal and external, over time.



2. Best Fit:

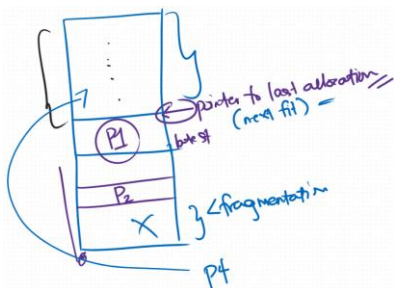
- **Description:** The best-fit approach allocates the smallest available block of memory that is large enough to fit the process.
- **Advantages:** Minimizes wasted memory by selecting the closest match in size. Reduces external fragmentation.
- **Disadvantages:** May lead to fragmentation and may require searching through the entire list of available blocks.

3. Worst Fit:

- **Description:** The worst-fit approach allocates the largest available block of memory. It aims to leave the largest holes for future allocations.
- **Advantages:** Can lead to larger available blocks for subsequent allocations.
- **Disadvantages:** May result in more wasted memory due to fragmentation. Like best fit, searching for the largest block can be time-consuming.

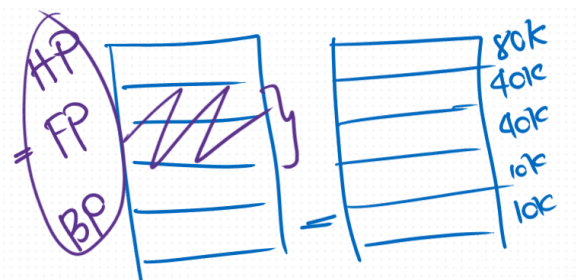
4. Next Fit:

- **Description:** Similar to the first-fit approach, but it starts searching for available memory from the last allocation point. It continues searching until a suitable block is found.
- **Advantages:** Reduces fragmentation compared to first fit. Simplifies implementation.
- **Disadvantages:** Still susceptible to fragmentation.



5. Quick Fit:

- **Description:** In quick fit, the memory is divided into predefined block sizes, and each size has its own list of available blocks. Allocation involves finding the appropriate size class and selecting the first available block.
- **Advantages:** Quick allocation, especially for commonly used block sizes.
- **Disadvantages:** May lead to fragmentation within size classes.



Each approach aims to reduce but cannot eliminate fragmentation.

Emulating a first-fit flat memory model

The necessary condition for a process to be selected for CPU execution is that an available memory block is available. If it is not available, we can either:

- Select a random process to be put into a backing store
- Put the candidate process back into the ready queue and wait for another chance to use the memory.

First, consider an OS design with **no backing store support** and use a **first-come-first-serve** scheduler.

Flat memory allocator example

```
108 //flat memory allocator with FCFS
109 #include <iostream>
110 #include <vector>
111 #include <unordered_map>
112
113 // Process class representing a process with name, id, and memory required
114 class Process {
115 public:
116     std::string name;
117     int id;
118     size_t memoryRequired;
119
120     Process(const std::string& name, int id, size_t memoryRequired)
121         : name(name), id(id), memoryRequired(memoryRequired) {}
122 };
123
124 // IMemoryAllocator interface
125 class IMemoryAllocator {
126 public:
127     virtual void* allocate(size_t size) = 0;
128     virtual void deallocate(void* ptr) = 0;
129     virtual std::string visualizeMemory() = 0;
130 };
131
```

The flat memory allocator is implemented as:

```
132 // FlatMemoryAllocator class implementing IMemoryAllocator
133 class FlatMemoryAllocator : public IMemoryAllocator {
134 public:
135     FlatMemoryAllocator(size_t maximumSize) : maximumSize(maximumSize), allocatedSize(0) {
136         memory.reserve(maximumSize);
137         initializeMemory();
138     }
139
140     ~FlatMemoryAllocator() {
141         memory.clear();
142     }
143
144     void* allocate(size_t size) override {
145         // Find the first available block that can accommodate the process
146         for (size_t i = 0; i < maximumSize - size + 1; ++i) {
147             if (!allocationMap[i] && canAllocateAt(i, size)) {
148                 allocateAt(i, size);
149                 return &memory[i];
150             }
151         }
152
153         // No available block found, return nullptr
154         return nullptr;
155     }
156
157     void deallocate(void* ptr) override {
158         // Find the index of the memory block to deallocate
159         size_t index = static_cast<char*>(ptr) - &memory[0];
160         if (allocationMap[index]) {
161             deallocateAt(index);
162         }
163     }
164
165     std::string visualizeMemory() override {
166         return std::string(memory.begin(), memory.end());
167     }
168 }
169
```

```

169 private:
170     size_t maximumSize;
171     size_t allocatedSize;
172     std::vector<char> memory;
173     std::unordered_map<size_t, bool> allocationMap;
174
175     void initializeMemory() {
176         std::fill(memory.begin(), memory.end(), '.'); // '.' represents unallocated memory
177         std::fill(allocationMap.begin(), allocationMap.end(), false);
178     }
179
180     bool canAllocateAt(size_t index, size_t size) const {
181         // Check if the memory block is large enough
182         return (index + size <= maximumSize);
183     }
184
185     void allocateAt(size_t index, size_t size) {
186         // Mark the memory block as allocated
187         std::fill(allocationMap.begin() + index, allocationMap.begin() + index + size, true);
188         allocatedSize += size;
189     }
190
191     void deallocateAt(size_t index) {
192         // Mark the memory block as deallocated
193         allocationMap[index] = false;
194     }
195 };

```

```

197 // Simple first-come-first-serve scheduler
198 class FCFS_Scheduler {
199 public:
200     FCFS_Scheduler(IMemoryAllocator& memoryAllocator) : memoryAllocator(memoryAllocator) {}
201
202     void addProcess(const Process& process) {
203         readyQueue.push_back(process);
204     }
205
206     void executeProcesses() {
207         for (const auto& process : readyQueue) {
208             void* memory = memoryAllocator.allocate(process.memoryRequired);
209
210             if (memory != nullptr) {
211                 std::cout << "Allocated memory for process " << process.name << " (ID: " << process.id << ")\n";
212                 std::cout << "Memory state: " << memoryAllocator.visualizeMemory() << "\n";
213                 memoryAllocator.deallocate(memory);
214             } else {
215                 std::cout << "Insufficient memory for process " << process.name << " (ID: " << process.id << ")\n";
216             }
217
218             //<additional instructions for executing the process in CPU is done here>
219         }
220
221         readyQueue.clear();
222     }
223
224 private:
225     IMemoryAllocator& memoryAllocator;
226     std::vector<Process> readyQueue;
227 };
228
229 int main() {
230     const size_t maximumMemorySize = 50;
231     FlatMemoryAllocator memoryAllocator(maximumMemorySize);
232     FCFS_Scheduler scheduler(memoryAllocator);
233
234     // Add processes to the scheduler
235     scheduler.addProcess(Process("Process_A", 1, 10));
236     scheduler.addProcess(Process("Process_B", 2, 5));
237     scheduler.addProcess(Process("Process_C", 3, 8));
238
239     // Execute processes
240     scheduler.executeProcesses();
241
242     return 0;
243 }

```

Concept of placement of new operator

- To effectively manage memory and have more control over which memory blocks are allocated, we use “placement new”
- Placement new is a variation new operator in C++.
 - Normal new operator does two things : (1) Allocates memory (2) Constructs an object in allocated memory.
 - Placement new allows us to separate above two things. In placement new, we can pass a preallocated memory and construct an object in the passed memory.

An example. Allocate 10 uint8_t items in a stack-based order:

```
109 void MemoryTestRun::testAllocation_uint8()
110 {
111     std::cout << "--TEST ALLOCATION. UINT8--" << std::endl;
112     const int MAX_SIZE = 10;
113
114     // Allocate a block of memory to store MAX_SIZE * sizeof(uint8_t) bytes
115     uint8_t* data = new uint8_t[MAX_SIZE * sizeof(uint8_t)];
116
117     // Pointer to keep track of the next available position (offset in bytes)
118     uint8_t* top = data;
119
120     // Push 10 values (treated as uint8_t) onto the stack using placement new
121     for (int i = 0; i < 10; ++i) {
122         new (top) uint8_t(200); // Placement new to construct uint8_t at top
123         top++;
124     }
125
126     // Access and print the elements (stack order - LIFO)
127     while (top != data) {
128         top--;
129         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
130     }
131
132     std::cout << std::endl;
133
134     // Deallocate the memory block (no placement delete needed)
135     delete[] data;
136 }
```

Another example. Allocate 10 int32_t items in a stack-based order:

```
138 void MemoryTestRun::testAllocation_int32()
139 {
140     std::cout << "--TEST ALLOCATION. UINT32--" << std::endl;
141
142     //ALLOCATION FOR SIGNED INT
143     const int MAX_SIZE = 10;
144
145     // Allocate a block of memory to store MAX_SIZE * sizeof(uint8_t) bytes
146     int32_t* data = new int32_t[MAX_SIZE * sizeof(int32_t)];
147
148     // Pointer to keep track of the next available position (offset in bytes)
149     int32_t* top = data;
150
151     // Push values onto the stack using placement new
152     for (int i = 0; i < 10; ++i) {
153         new (top) int32_t(15235521); // Placement new to construct uint8_t at top
154         top++;
155     }
156
157     // Access and print the elements (stack order - LIFO)
158     while (top != data) {
159         top--;
160         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
161     }
162
163     std::cout << std::endl;
164
165     // Deallocate the memory block (no placement delete needed)
166     delete[] data;
167 }
168 }
```

Below shows the sample run of these functions:

```
--TEST ALLOCATION. UINT8--
Value: 200, Address: 000002A2CBC27D49
Value: 200, Address: 000002A2CBC27D48
Value: 200, Address: 000002A2CBC27D47
Value: 200, Address: 000002A2CBC27D46
Value: 200, Address: 000002A2CBC27D45
Value: 200, Address: 000002A2CBC27D44
Value: 200, Address: 000002A2CBC27D43
Value: 200, Address: 000002A2CBC27D42
Value: 200, Address: 000002A2CBC27D41
Value: 200, Address: 000002A2CBC27D40

--TEST ALLOCATION. UINT32--
Value: 15235521, Address: 000002A2CB8CEE44
Value: 15235521, Address: 000002A2CB8CEE40
Value: 15235521, Address: 000002A2CB8CEE3C
Value: 15235521, Address: 000002A2CB8CEE38
Value: 15235521, Address: 000002A2CB8CEE34
Value: 15235521, Address: 000002A2CB8CEE30
Value: 15235521, Address: 000002A2CB8CEE2C
Value: 15235521, Address: 000002A2CB8CEE28
Value: 15235521, Address: 000002A2CB8CEE24
Value: 15235521, Address: 000002A2CB8CEE20
```

Explanation of the uint8_t and int32_t placement new allocation:

Allocation:

- Allocates a block of memory to store 10 times the size of a uint8_t using new uint8_t[MAX_SIZE * sizeof(uint8_t)].

Stack Implementation:

- Uses a uint8_t* pointer named top to keep track of the next available position, mimicking a stack.

Pushing Elements (10 times):

- Iterates 10 times.
- Inside the loop:
 - Uses new (top) uint8_t(200) to construct a uint8_t with value 200 at the memory location pointed to by top (placement new).
 - Increments top to point to the next available position.

Printing Elements (LIFO order):

- Loops until top reaches the beginning of the allocated memory.
- Inside the loop:
 - Decrements top to point to the last element pushed.
 - Prints the value of the element at *top after casting it to int for easier visualization (since uint8_t is typically printed as an unsigned char).
 - Prints the memory address of the element using reinterpret_cast<void*>(top).

Deallocation:

- Deallocates the entire memory block using delete[] data.

There may be cases where we may need to **reuse** the memory allocated. This is where the “placement new” operator is needed, as there is no need to allocate new memory.

Below is an example of our allocation using 10 `int32_t` items, which are then replaced by new items.

```
170 void MemoryTestRun::testReAllocation_int32()
171 {
172     std::cout << "--TEST ALLOCATION WITH OVERWRITING. INT32--" << std::endl;
173
174     //ALLOCATION FOR SIGNED INT
175     const int MAX_SIZE = 10;
176
177     // Allocate a block of memory to store MAX_SIZE * sizeof(int32_t) bytes
178     int32_t* data = new int32_t[MAX_SIZE * sizeof(int32_t)];
179
180     // Pointer to keep track of the next available position (offset in bytes)
181     int32_t* top = data;
182
183     // Push values onto the stack using placement new
184     for (int i = 0; i < 10; ++i) {
185         new (top) int32_t(15235521); // Placement new to construct int32 at top
186         top++;
187     }
188
189     // Access and print the elements (stack order - LIFO)
190     while (top != data) {
191         top--;
192         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
193     }
194     std::cout << std::endl;
195     std::cout << "Re-allocating/overwriting data!" << std::endl;
196
197     //reallocate new data
198     //Push values onto the stack using placement new
199     for (int i = 0; i < 10; ++i) {
200         new (top) int32_t(-100); // Placement new to construct int32 at top
201         top++;
202     }
203
204     // Access and print the elements (stack order - LIFO)
205     while (top != data) {
206         top--;
207         std::cout << "Value: " << static_cast<int>(*top) << ", Address: " << reinterpret_cast<int>(top) << std::endl;
208     }
209
210     std::cout << std::endl;
211
212     // Deallocate the memory block (no placement delete needed)
213     delete[] data;
214 }
215 }
```

Explanation:

1. Allocation for Signed Integers (`int32_t`):
 - Allocates a block of memory to store 10 times the size of an `int32_t` using `new int32_t[MAX_SIZE * sizeof(int32_t)]`.
 - Creates a pointer `top` of type `int32_t*` to keep track of the next available position.
2. Pushing Initial Values (10 times):
 - Iterates 10 times.
 - Inside the loop:
 - Uses `new (top) int32_t(15235521)` to construct an `int32_t` with value 15235521 at the memory location pointed to by `top` (placement new).
 - Increments `top` to point to the next available position.
3. Printing Initial Elements (LIFO order):
 - Loops until `top` reaches the beginning of the allocated memory.
 - Inside the loop:
 - Decrements `top` to point to the last element pushed.
 - Prints the value of the element at `*top` after casting it to `int` for easier visualization.
 - Prints the memory address of the element using `reinterpret_cast<int>(top)`.
4. Re-allocating/Overwriting Data:
 - Prints a message indicating re-allocation and overwriting.
5. Overwriting Existing Values (10 times):
 - Iterates 10 times again.
 - Inside the loop:
 - Uses `new (top) int32_t(-100)` to construct an `int32_t` with value -100 at the same memory locations used previously (overwriting existing values).
 - Increments `top` to point to the next available position (although it's not actually used in this case).
6. Printing Overwritten Elements (LIFO order):
 - Loops until the `top` reaches the beginning of the allocated memory.
 - Inside the loop:

- Decrements top to point to the last overwritten element.
- Prints the value (now -100) and memory address as before.

7. Deallocation:

- Deallocates the entire memory block using delete[] data. This frees the memory even though the data was overwritten.

Purpose of placement new with OS memory management

In C++ (especially in OS code where structures can be found at fixed addresses), it can be useful to construct an object in memory that is obtained elsewhere. This is accomplished through a technique known as 'placement new'. For example, say you wanted to create an APIC object at address 0x09FFF0000, then this snippet of code will use placement new to do the trick:

```
void *apic_address = reinterpret_cast<void *>(0x09FFF0000);
APIC *apic = new (apic_address) APIC;
```

Device driver buffers:

- Device drivers might pre-allocate buffers for communication with hardware. Placement new could be used to construct data structures (like headers) within these buffers to manage the data.
- Without placement new, N device driver buffers may be placed at different locations in memory
- Harder to debug, harder to visualize.



Process blocks:

- This is useful for efficiently dividing process memory into its defined components: stack, heap, symbol table, etc.

Fixed memory address range for user processes:

- We can define a fixed memory block where N processes get stored – the user process space vs the OS space.

Purpose of “placement new” in the flat memory model:

- In our previous example, since we’re tracking memory blocks in data structures like vectors and accessing them via unordered map, “placement new” could be determined for identifying the “starting points” for each block, or a “single entry point” for the whole memory block.
- “Placement new” could be used for more granular tracking of memory addresses → becomes more manual → pointer to available memory, plus using sizeof() for allocation.

Activity:

- Show a working application emulating the first-fit memory allocator for a main memory with $2^{14} = 16,384$ bytes of memory.
 - Set the following configuration for your latest OS emulator:
 - num-cpu 2
 - scheduler “rr”
 - quantum-cycles 4
 - preemptive 1
 - batch-process-freq 0.25
 - min-ins 100
 - max-ins 100

- Set the newest parameters for your OS emulator memory manager:
 - max-overall-mem 16384
 - min-mem-per-proc 16
 - max-mem-per-proc 4096
- Each process will have a random amount of memory required: $2^4(16) \leq M \leq 2^{12}(4096)$.
Note: This is your “min-mem-per-proc” and “max-mem-per-proc” in your MC02 specs.
- Each process stays in the memory during its execution. It will be released from memory every time it is pre-empted/finished execution.
- If memory is full when a process is scheduled, it will revert back to the tail of the ready queue.
- For every quantum-cycles, produce a text file with the following information:
 - Timestamp
 - Number of processes in memory
 - Total external fragmentation in KB
 - An ASCII printout of the memory.
 - See mockup below:

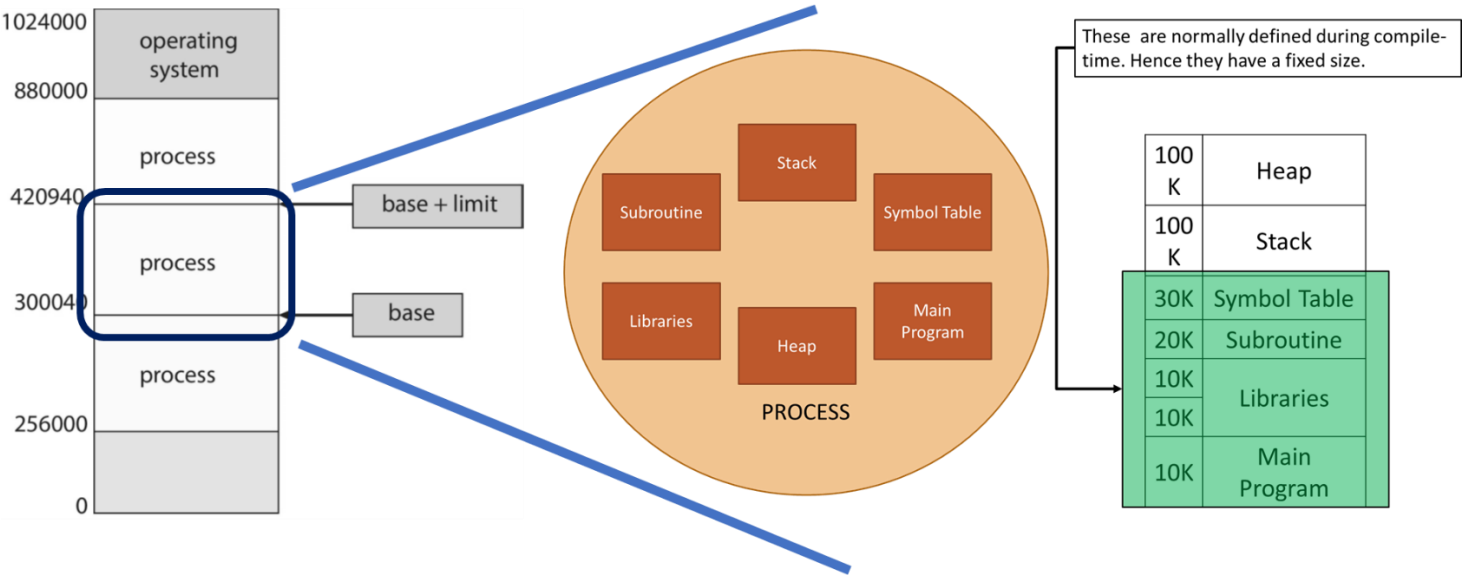

```

168   Timestamp: (08/06/2024 09:15:22AM)
169   Number of processes in memory: 2
170   Total external fragmentation in KB: 8192
171
172   ----end---- = 16384
173
174   16384
175   P1
176   12288
177
178   8192
179   P9
180   4096
181
182   ----start---- = 0
          
```
 - Explanation: The boundaries indicate the memory address limits. For every process, print its upper limit, and lower limit, following the prescribed layout. In this example, there are only 2 processes in memory. Then, there's external fragmentation of: $12,288 (P1 \text{ lower bound}) - 8,192 (P2 \text{ upper bound}) + 4096 (P2 \text{ lower bound}) - 0 (\text{start of address}) = 8,192$ bytes.

Paging

- Most OS paging schemes.
- This method allows non-contiguous allocation of memory for processes → although internal fragmentation may still occur.

Recall that a process is stored in memory and divided into subspaces.



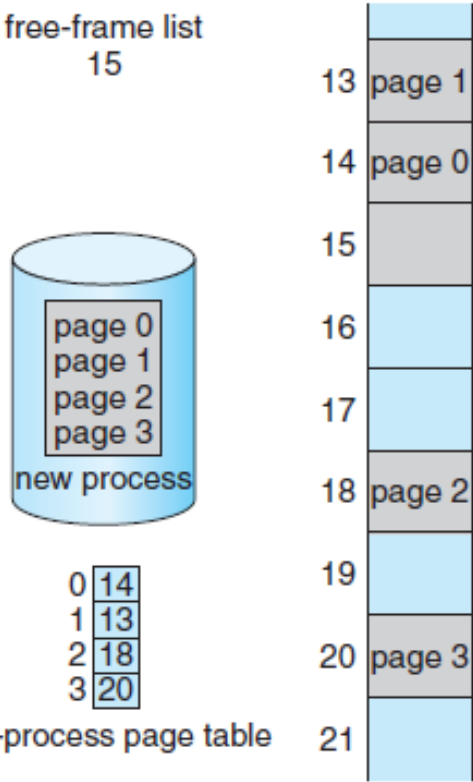
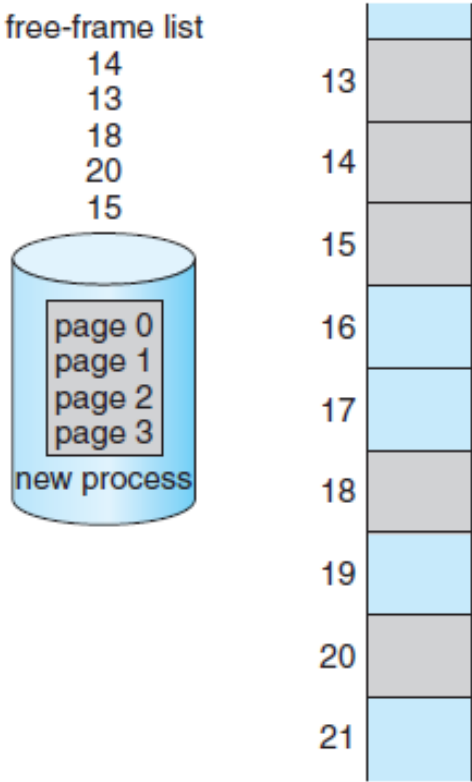
Paging Concepts

The subspaces in each processes are referred to as **pages**.

100 K	Heap	0
100 K	Stack	1
30K	Symbol Table	2
20K	Subroutine	3
10K	Libraries	4
10K		
10K	Main Program	5

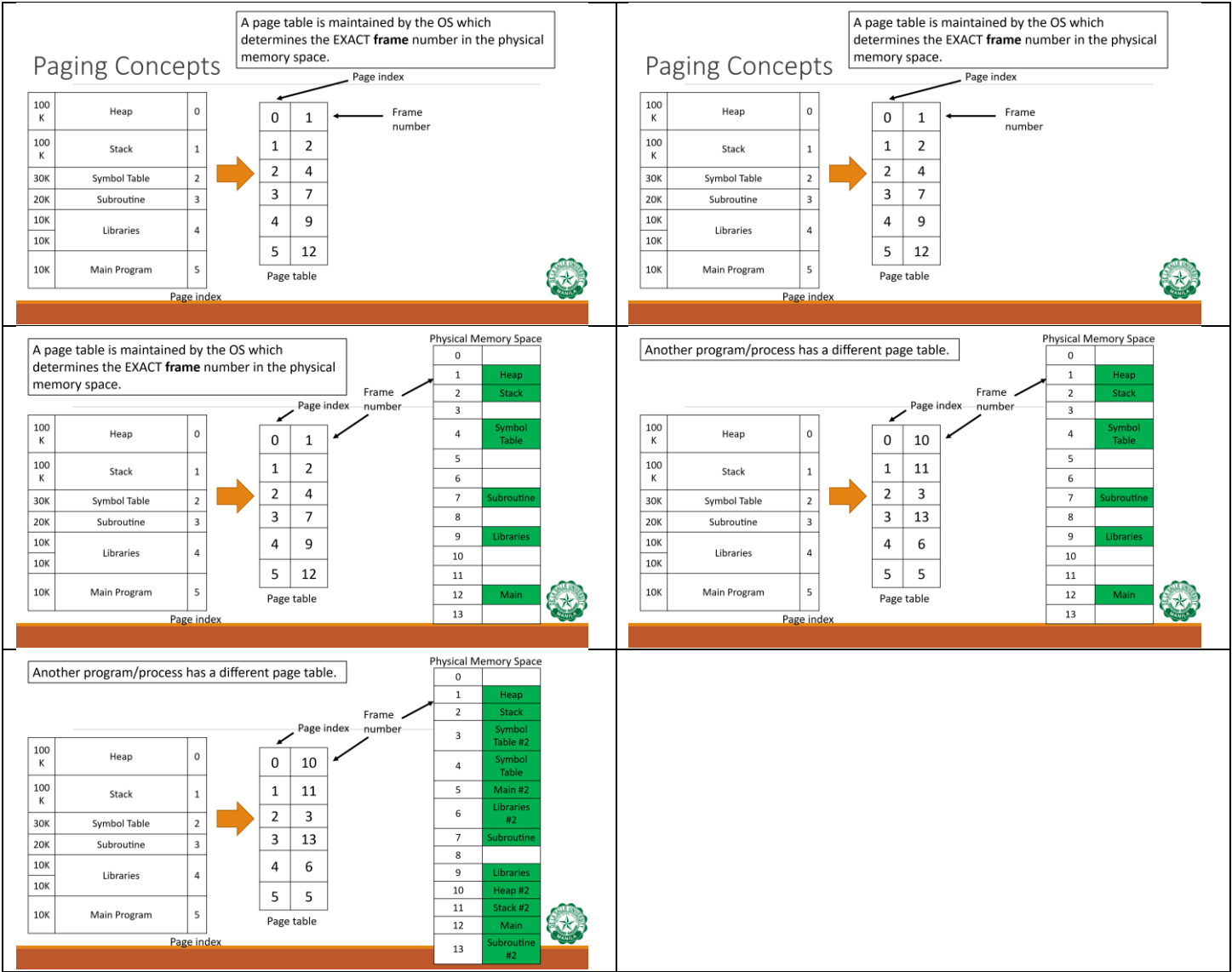
1. The OS maintains a list of free frames.
2. All processes that arrive have a set number of pages.
3. The OS selects free frame lists to assign the pages and creates the page table.
4. Processes that finishes execution returns the free frames.

This is illustrated in the following diagram:



(a)

(b)



Paging Problem #1

5. Consider processes wherein each need **5 pages** to execute. The OS has 15 frames available and the **free-frame** list is treated as a queue and has the following values: 13,14,15,4,5,6,7,8,0,1,2,3,9,10,12,11. If the **free-frame** list is empty, the process will need to wait. Consider the process table and using a CPU scheduling scheme of **pre-emptive shortest-job first**, answer the following questions.

Process	Arrival Time	CPU Burst
P1	0	10
P2	1	8
P3	2	8
P4	3	3
P5	4	3

- A. Draw the **physical memory map** by time slice 10.
- B. What will be the last value of the **free-frame** list after all processes have finished executing?
- C. Indicate **all free frames** by time slice 10.
- D. Draw the **physical memory map** by time slice 15.

Paging Problem #2

6. Consider processes wherein each need **5 pages** to execute. The OS has 10 frames available and the **free-frame** list is treated as a queue and has the following values: 4,5,6,7,0,1,2,3,8,9. The OS has **swapping enabled** and performs swapping on the oldest process. Consider the process table and using a CPU scheduling scheme of **pre-emptive shortest job first**, answer the following questions.

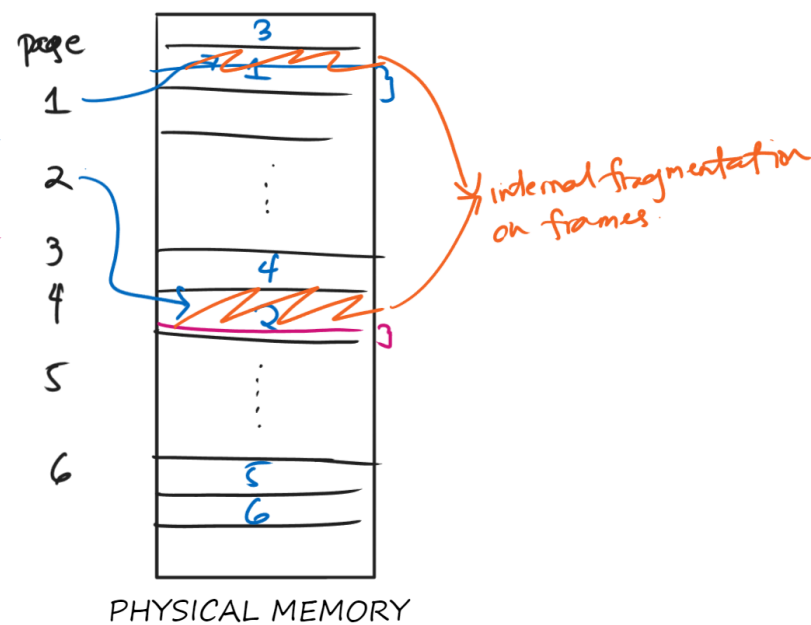
Process	Arrival Time	CPU Burst
P1	0	10
P2	5	5
P3	10	13
P4	12	10
P5	14	5

- A. What will be the last value of the **free-frame** list after all processes have finished executing?
- B. Indicate the allocated **frames** for P1.
- C. Indicate the allocated **frames** for P2.
- D. Indicate the allocated **frames** for P3.
- E. Indicate the allocated **frames** for P4.
- F. Indicate the allocated **frames** for P5.

Internal fragmentation only happens within the pages themselves but they are very minimal → heap and stack pre-allocations.

Consider a running process

100K	Heap	} ← currently used space
100K	Stack	
30K	Symbol Table	← currently used space
20K	Subroutine	
10K	Libraries	
10K		
10K	Main Program	



Emulating paging and backing store

Code translations:

- The process, memory allocator, and scheduling classes discussed previously can be extended to support paging.
- Paging → a process must have defined page numbers. Following the components of a program, we could design each process to have **six** pages with the following indices:

Page index	Purpose
0	Heap
1	Stack
2	Symbol table
3	Subroutine
4	Libraries
5	Main program

- For simplicity, we could limit each page to a fixed sized → placeholder sizeof(char) or sizeof(int). We are merely simulating the memory usage of each page.
- A class must hold the free frame list and how each frame is mapped to a memory block → PagingAllocator
- The paging allocator must compute the ideal number of frames to be allocated to a given memory size → the physical memory block.

Sample code template:

```
265 class IMemoryAllocator {
266 public:
267     virtual void* allocate(Process* process) = 0;
268     virtual void deallocate(Process* process) = 0;
269     virtual void visualizeMemory() const = 0;
270 };
271
272 class PagingAllocator : public IMemoryAllocator {
273 public:
274     PagingAllocator(size_t maxMemorySize);
275
276     void* allocate(Process* process) override;
277     void deallocate(Process* process) override;
278     void visualizeMemory() const override;
279
280 private:
281     size_t maxMemorySize;
282     size_t numFrames;
283     std::unordered_map<size_t, size_t> frameMap;
284     std::vector<size_t> freeFrameList;
285
286     size_t allocateFrames(size_t numFrames, size_t processId, const std::vector<size_t>& pageSizes);
287     void deallocateFrames(size_t numFrames, size_t frameIndex, const std::vector<size_t>& pageSizes);
288 };
```

The paging allocator implementation

```

324 // Implementation of PagingAllocator class
325 PagingAllocator::PagingAllocator(size_t maxMemorySize)
326 : maxMemorySize(maxMemorySize), numFrames(maxMemorySize) {
327     // Initialize the free frame list
328     for (size_t i = 0; i < numFrames; ++i) {
329         freeFrameList.push_back(i);
330     }
331 }
332
333 void* PagingAllocator::allocate(Process* process) {
334     size_t processId = process->getId();
335     size_t numFramesNeeded = process->getNumPages();
336     if (numFramesNeeded > freeFrameList.size()) {
337         std::cerr << "Memory allocation failed. Not enough free frames.\n";
338         return nullptr;
339     }
340
341     // Allocate frames for the process
342     size_t frameIndex = allocateFrames(numFramesNeeded, processId);
343     return reinterpret_cast<void*>(frameIndex);
344 }
345
346 void PagingAllocator::deallocate(Process* process) {
347     size_t processId = process->getId();
348
349     // Find frames allocated to the process and deallocate
350     auto it = std::find_if(frameMap.begin(), frameMap.end(),
351         [processId](const auto& entry) { return entry.second == processId; });
352
353     while (it != frameMap.end()) {
354         size_t frameIndex = it->first;
355         deallocateFrames(1, frameIndex);
356         it = std::find_if(frameMap.begin(), frameMap.end(),
357             [processId](const auto& entry) { return entry.second == processId; });
358     }
359 }
360
361 void PagingAllocator::visualizeMemory() const {
362     std::cout << "Memory Visualization:\n";
363     for (size_t frameIndex = 0; frameIndex < numFrames; ++frameIndex) {
364         auto it = frameMap.find(frameIndex);
365         if (it != frameMap.end()) {
366             std::cout << "Frame " << frameIndex << " -> Process " << it->second << "\n";
367         } else {
368             std::cout << "Frame " << frameIndex << " -> Free\n";
369         }
370     }
371     std::cout << "-----\n";
372 }
373
374 size_t PagingAllocator::allocateFrames(size_t numFrames, size_t processId, const std::vector<size_t>& pageSizes) {
375     size_t frameIndex = freeFrameList.back();
376     freeFrameList.pop_back();
377
378     // Map allocated frames to the process ID
379     for (size_t i = 0; i < numFrames; ++i) {
380         frameMap[frameIndex + i] = processId;
381     }
382
383     return frameIndex;
384 }
385
386 void PagingAllocator::deallocateFrames(size_t numFrames, size_t frameIndex, const std::vector<size_t>& pageSizes) {
387     // Remove mapping of deallocated frames
388     for (size_t i = 0; i < numFrames; ++i) {
389         frameMap.erase(frameIndex + i);
390     }
391
392     // Add frames back to the free frame list
393     for (size_t i = 0; i < numFrames; ++i) {
394         freeFrameList.push_back(frameIndex + i);
395     }
396 }

```

Consider a sample paging scheduler (FCFS scheduler) below that uses the paging allocator:

```

407 void PagingScheduler::addProcess(Process* process) {
408     // Allocate memory for the process
409     void* memoryBlock = memoryAllocator.allocate(process);
410     if (!memoryBlock) {
411         std::cerr << "Failed to add process " << process->getId() << ". Memory allocation failed.\n";
412         return;
413     }
414
415     size_t coreId = rand() % numCores; // Assign the process to a random core
416     processCores[process] = coreId;
417     process->setMemoryBlock(memoryBlock);
418     std::cout << "Process " << process->getId() << " added to Core " << coreId << ".\n";
419 }
420
421 void PagingScheduler::removeProcess(Process* process) {
422     // Deallocate memory for the process
423     memoryAllocator.deallocate(process);
424     processCores.erase(process);
425     std::cout << "Process " << process->getId() << " removed.\n";
426 }
427
428 void PagingScheduler::visualizeMemory() const {
429     memoryAllocator.visualizeMemory();
430 }
431
432 void PagingScheduler::simulateExecution() {
433     for (auto& thread : coreThreads) {
434         thread.join();
435     }
436 }
437
438 void PagingScheduler::coreFunction(size_t coreId) {
439     <do some CPU work>
440 }

```

Sample usage:

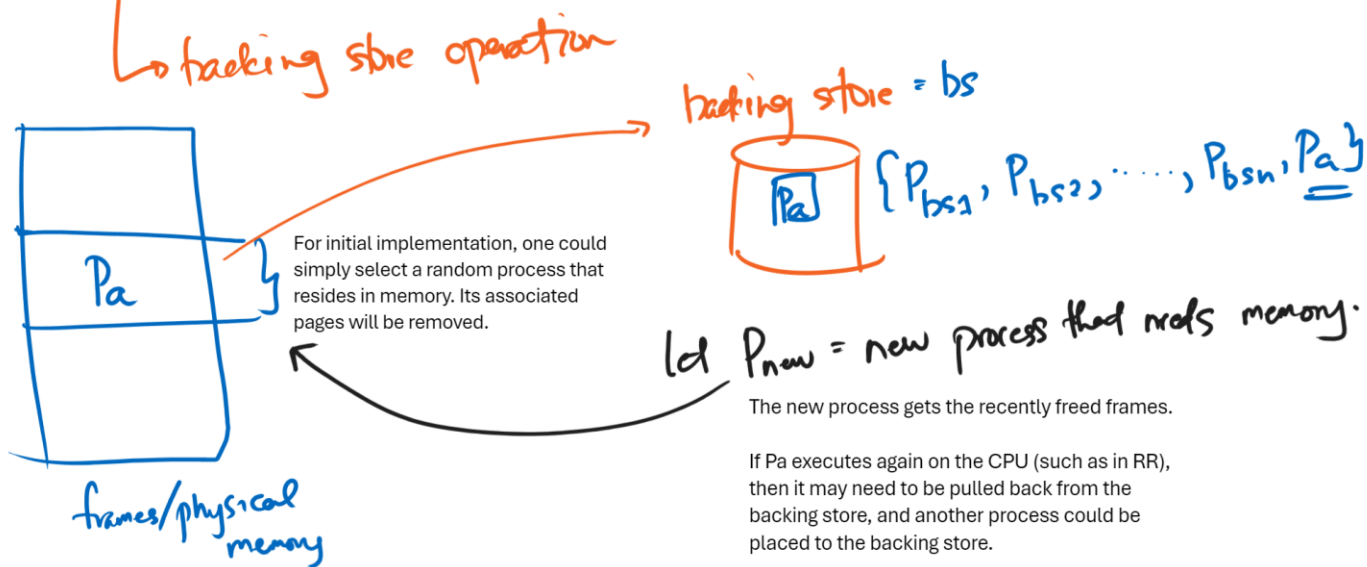
```

442 int main() {
443     // Create processes with specified page sizes
444     Process process1(1, {256, 256, 256, 256, 256});
445     Process process2(2, {256, 256, 256, 256, 256});
446     Process process3(3, {256, 256, 256, 256, 256});
447     Process process4(4, {256, 256, 256, 256, 256});
448     Process process5(5, {256, 256, 256, 256, 256});
449
450     // Set up paging scheduler with 4 cores and maximum memory size
451     PagingScheduler scheduler(4, 4096);
452
453     // Add processes to the scheduler
454     scheduler.addProcess(&process1);
455     scheduler.addProcess(&process2);
456     scheduler.addProcess(&process3);
457     scheduler.addProcess(&process4);
458     scheduler.addProcess(&process5);
459
460     // Visualize initial memory state
461     scheduler.visualizeMemory();
462
463     // Simulate process execution
464     scheduler.simulateExecution();
465
466     // Processes automatically deallocated upon destruction
467
468     return 0;
469 }

```

Backing store operation

```
407 void PagingScheduler::addProcess(Process* process) {
408     // Allocate memory for the process
409     void* memoryBlock = memoryAllocator.allocate(process);
410     if (!memoryBlock) {
411         std::cerr << "Failed to add process " << process->getId() << ". Memory allocation failed.\n";
412         return;
413     }
```



To emulate a backing store:

- A backing store could be text files saved in a certain directory. E.g. a backing store directory relative to your CSOPESY emulator path.
- Why must we save to text files? We must free our **main memory** usage.
- What should we save in our text files? All necessary process info must be saved, such as:
 - Process id, name, command counter
 - The number of pages and associated memory sizes, as dictated by its last command execution. In our case, since process memory and page size is fixed, we simply store these values.

Backing store architectural concept and real-world alignment

- Backing store (or swap space) is typically implemented as a dedicated disk partition (e.g., Linux swap partition or Windows pagefile.sys)
 - Real-World OS Strategy: When physical RAM frames run low, the OS memory manager uses block-layer I/O calls to write victim pages directly to disk sectors. It tracks these pages using a swap map or swap offsets within the Page Table Entry (PTE).
 - OS Emulator Strategy: In our command-line emulator, we simulate physical RAM using a centralized byte array or frame pool. To mimic secondary storage transparently, we serialize paged-out data into a single persistent text file: csopesy-backing-store.txt.
- Core Challenges in Implementation
 - State preservation
 - When a page is evicted, we must not only save the raw bytes and variables but also maintain the context. E.g. process ID, page index that contains the data, symbol table values, last instruction line, and so on → the OS must be able to resume the process seamlessly.
 - Serialization
 - Values, data types (assuming we support beyond uint16), and different code instructions must be saved in a parseable format.

An example of a structural design of a backing store

The emulator's backing store can be designed using an Indexed Record Architecture. Every time a page is paged out, it is assigned an address format or text block delimiter in the file.

```
[PID: 1][PAGE: 0][TYPE: SYMBOL_TABLE]
varA:10;varB:5;
[PID: 1][PAGE: 1][TYPE: MAIN_PROGRAM]
DECLARE varA 10;WRITE 0x500 varA;
[PID: 2][PAGE: 0][TYPE: SYMBOL_TABLE]
varC:42;
```

Every read operation from this file must increment the “num-paged-in” counter and every write must increment the “num-page-out” counter, which will be reflected in the vmstat command.

Data structure

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <unordered_map>
#include <sstream>

// Representation of a virtual page's current status
struct PageTableEntry {
    int frameNumber = -1; // Maps to physical RAM if valid
    bool isValid = false; // true = in RAM, false = in Backing Store (Page Fault target)
    bool isDirty = false; // true = modified since loaded, requires write-back
};

// Simulated page data block (can hold variables or instructions)
struct PageData {
    int pageIndex;
    std::string type; // e.g., "SYMBOL_TABLE", "MAIN_PROGRAM"
    std::string rawContent; // Serialized string representation of the variables/instructions
};
```

Other notes

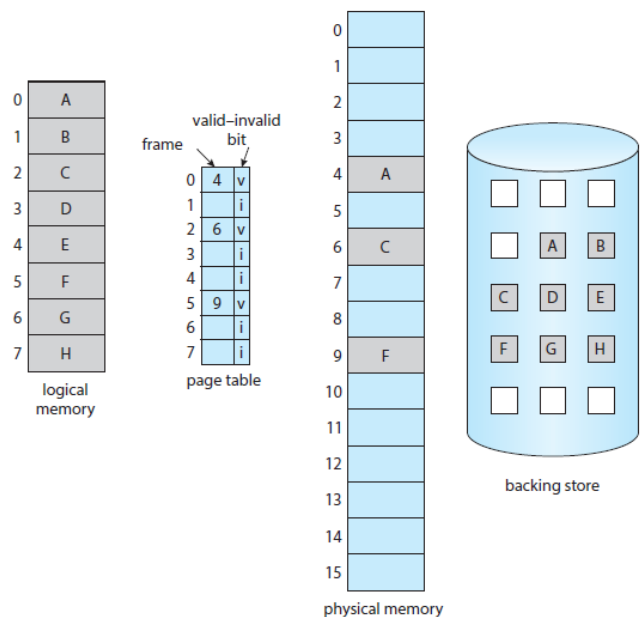
- The movement between the “memory representation” in the running emulator, and to the backing store (csopesy-backing-store.txt) must happen automatically underneath, without intervention from the user.
- Locality of reference – the execution must pause, loop inside a “page fault” phase until the process state is loaded from the backing store successfully. Instructions are restarted flawlessly, if necessary.

Demand paging

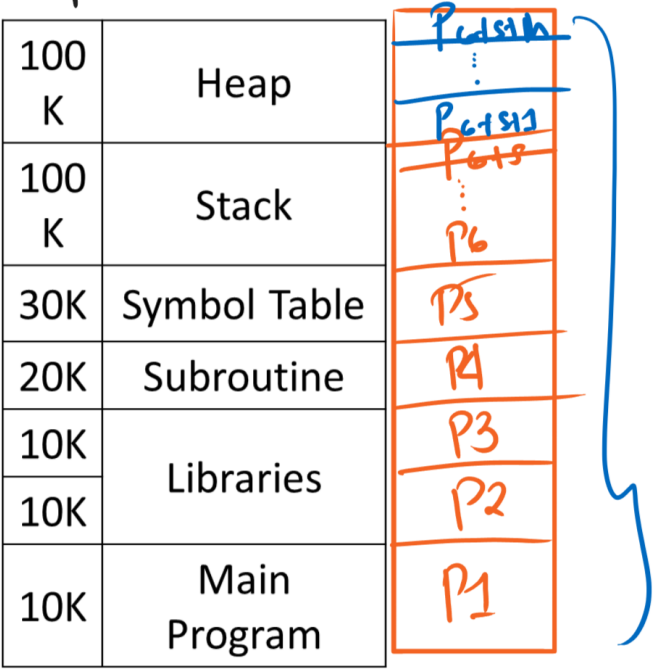
- To further conserve and have a more efficient memory use, we must ensure that the process/es selected for CPU execution already reside in our main memory.
- To ensure that we have free frames as much as possible, an alternative strategy is to load pages only as they are needed - demand paging.

Basic concepts

- The page table is extended to indicate a valid/invalid bit. If the page index is marked as valid, it means it is in main memory. If it is invalid, it resides in the backing store.
- At the start of execution, an active process will have **all its pages** in the backing store and marked as invalid.
- At any point in execution, an active process can have **some of its pages** put to main memory, and some put back into a backing store if it's not needed → another active process can then load its needed pages into main memory.



Demand paging



We can fine-grain the required pages per process, where subcomponents may require multiple pages.

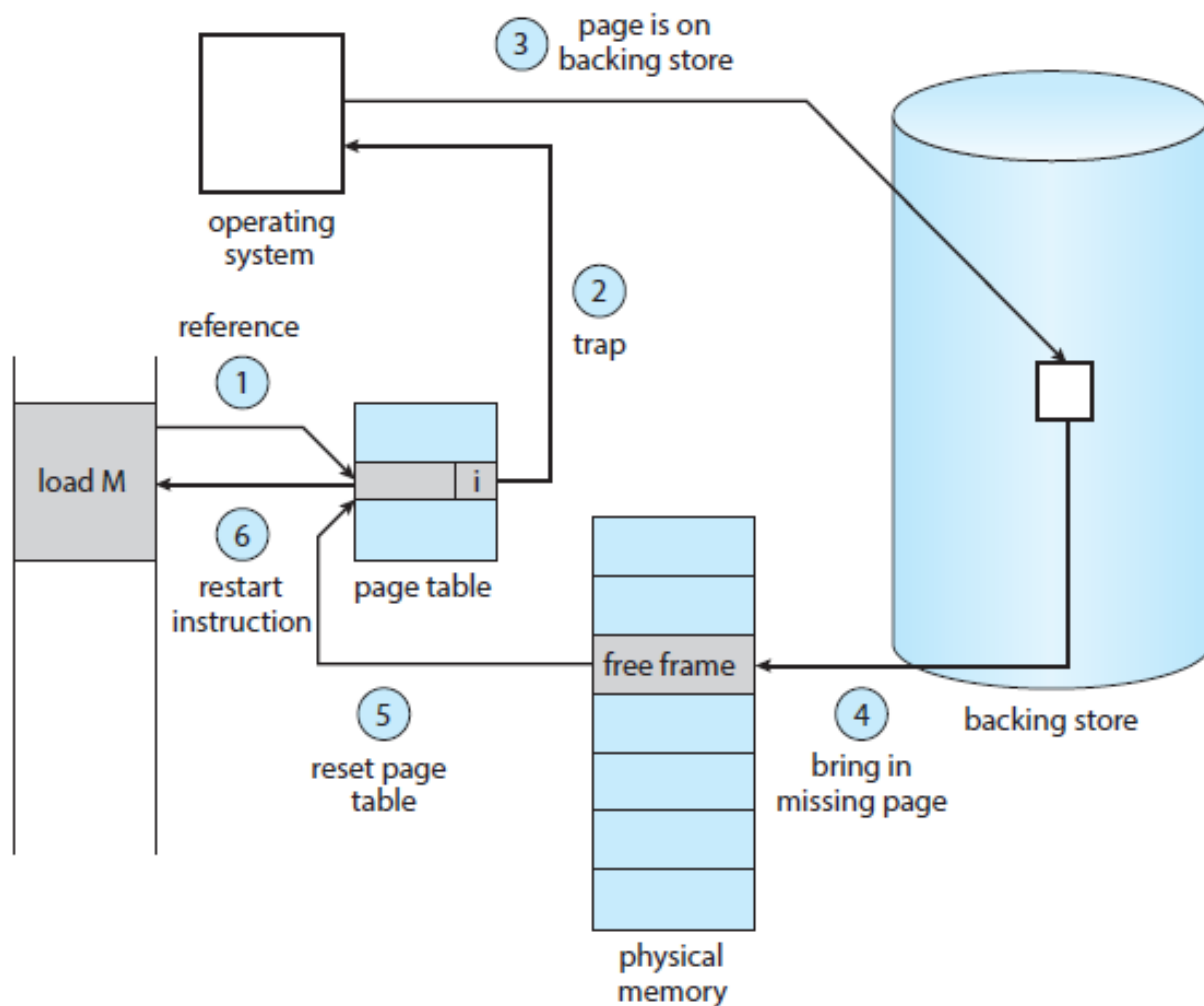
Purpose: Allows processes with smaller memory footprint to run with other processes that need larger memory.

The compiler and/or OS typically decides for the page allocation of each process.

Paging instructions and page fault

- If some pages are in a backing store, then this implies that a process may inevitably access a page that is not yet in main memory.
- Such incident is called a **page fault** - access to a page marked invalid.

The procedure for handling page faults are as follows (see illustration) :



1. We check an internal table (page table) for this process to determine whether the reference was a valid or invalid memory access.
2. If the reference is invalid, we terminate the process (access violation error. This happens in system-level programming languages where memory allocation is explicit). If it was valid but we have not yet brought in that page, we now page it in.
3. We find a free frame.
4. We schedule a secondary storage operation to read the desired page into the newly allocated frame.
5. When the storage read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted. The process can now access the page as though it had always been in memory.

Theoretically, some programs could access several new pages of memory with each instruction execution (one page for the instruction and many for data), possibly causing multiple page faults per instruction. This situation would result in unacceptable system performance. Fortunately, analysis of running processes shows that this behavior is exceedingly unlikely. Programs tend to have locality of reference which results in reasonable performance from demand paging.

The hardware to support demand paging is the same as the hardware for paging and swapping:

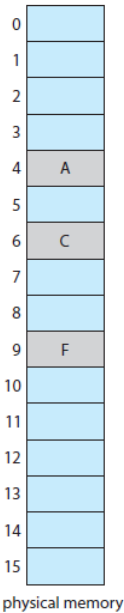
- **Page table.** This table has the ability to mark an entry invalid through a valid-invalid bit or a special value of protection bits.
- **Secondary memory.** This memory holds those pages that are not present in the main memory. The secondary memory is usually a high-speed disk or NVM device. It is known as the swap device, and the section of storage used for this purpose is known as swap space/backing store.

Maintaining free frames

- Most OS maintain a free-frame list, a pool of free frames for satisfying such requests.
- Free frames are typically allocated when the stack or heap segments from a process expand.



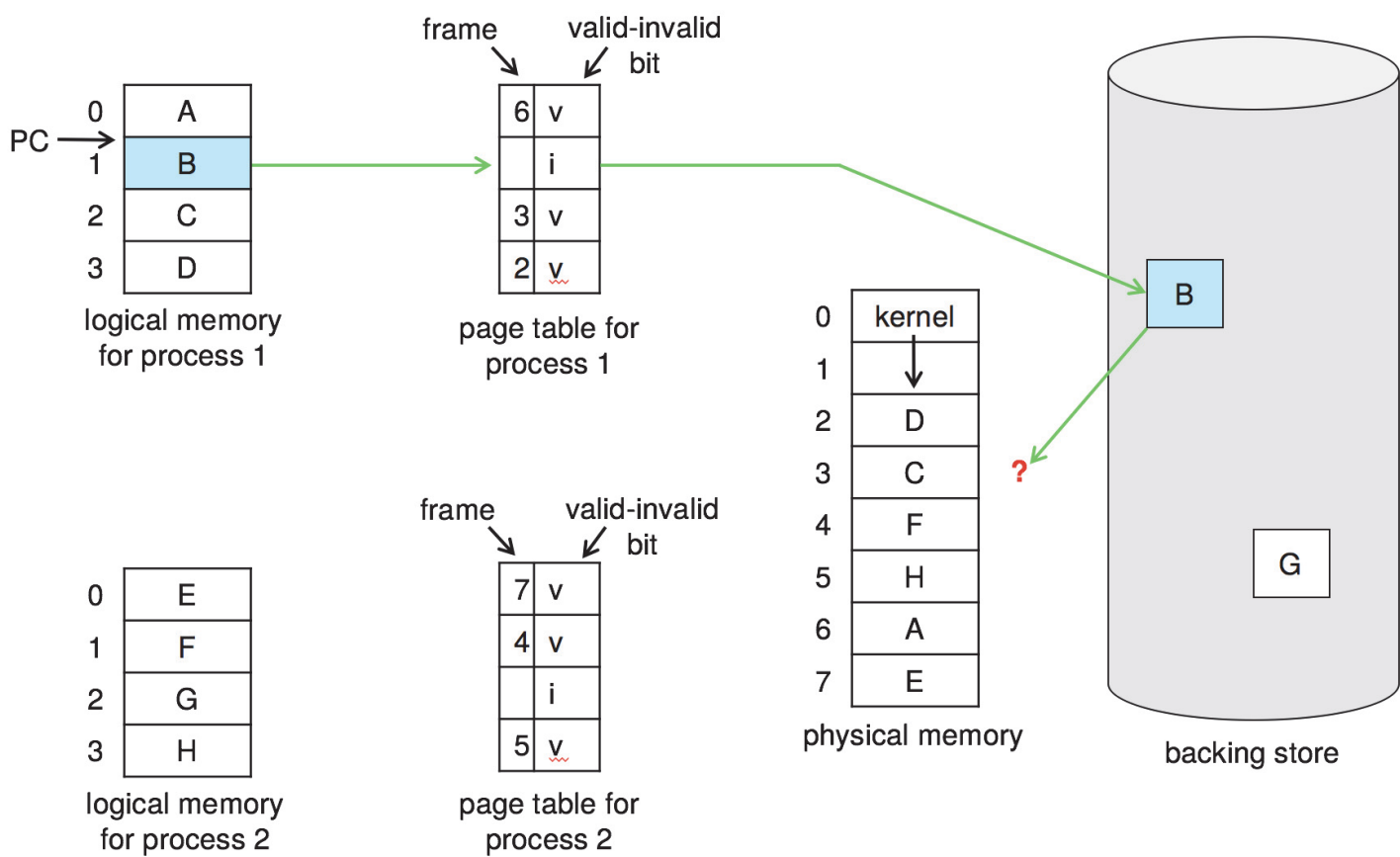
- When a system starts up, all available memory is placed on the free-frame list. As free frames are requested (for example, through demand paging), the size of the free-frame list shrinks. At some point, the list either falls to zero or falls below a certain threshold, at which point it must be repopulated - page replacement.



Page replacement

- As OS tries to run multiple processes, the degree of multi-programming increases → high risk and frequency of running out of free frames.
- The operating system has several options at this point:
 - Terminate the process
 - Easy to implement, but not good for the user.
 - Demand paging is the operating system’s attempt to improve the computer system’s utilization and throughput.
 - Users should not be aware that their processes are running on a paged system— paging should be logically transparent to the user. So, this option is not the best choice.
 - Perform page replacement → if no frame is free, then find a candidate page being used and free it - but which page?

Page replacement is illustrated below:



The general algorithm for **page replacement** is as follows:

1. Find the location of the desired page on secondary storage.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to secondary storage; change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and frame tables.
4. Continue the process from where the page fault occurred.

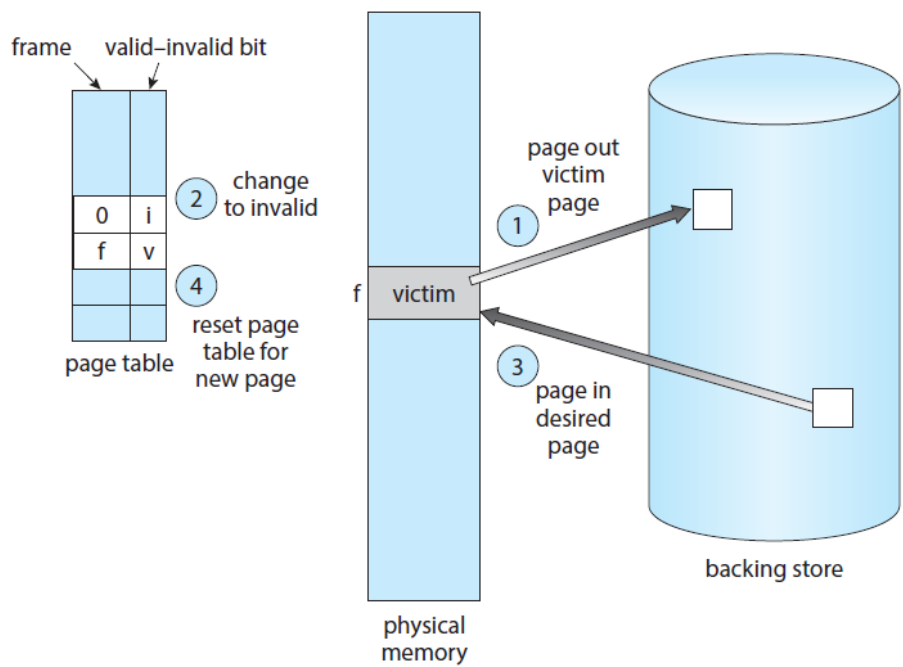


Figure 10.10 Page replacement.

With page replacement, we technically have infinite virtual memory

- Memory seen by the OS and processes isn't directly tied to the limits of the physical memory.

- Assume that there are no free frames, OS will always find a victim frame to replace
- There are different page replacement algorithms:

Random:

- **Principle:** Randomly selects a page to evict.
- **Pros:** Simplicity.
- **Cons:** Unpredictable and may not perform well in practice.

FIFO (First-In-First-Out):

- **Principle:** Evict the oldest page in the memory queue.
- **Pros:** Simple and easy to implement.
- **Cons:** May suffer from the "Belady's Anomaly," where increasing the number of frames does not guarantee a reduction in page faults.

Optimal Page Replacement:

- **Principle:** Evict the page that will not be used for the longest period (optimal, but not practically implementable).
- **Pros:** Theoretical optimal solution.
- **Cons:** Requires future knowledge of page accesses, which is not feasible in a real system.

LRU (Least Recently Used):

- **Principle:** Evict the page that has not been used for the longest time.
- **Pros:** Intuitive and often effective.
- **Cons:** Implementation complexity. Keeping track of usage timestamps can be resource-intensive.

```

478 // Process class (Assuming each page has a fixed size)
479 class Process {
480 public:
481     Process(int id, std::vector<int> pageIndices) : id(id), pageIndices(pageIndices) {}
482
483     int getId() const { return id; }
484     const std::vector<int>& getPageIndices() const { return pageIndices; }
485
486 private:
487     int id;
488     std::vector<int> pageIndices;
489 };
490
491 // FIFO Page Replacement Algorithm
492 class FIFOPageReplacement {
493 public:
494     FIFOPageReplacement(int frameCount) : frameCount(frameCount) {}
495
496     void loadProcess(const Process& process) {
497         std::cout << "Loading Process " << process.getId() << " into memory...\n";
498
499         for (int pageIndex : process.getPageIndices()) {
500             if (pageTable.find(pageIndex) == pageTable.end()) {
501                 // Page fault: Page is not in memory
502                 if (memoryQueue.size() >= frameCount) {
503                     // Evict the oldest page if memory is full
504                     int oldestPage = memoryQueue.front();
505                     memoryQueue.pop();
506                     pageTable.erase(oldestPage);
507                     std::cout << "Evicting Page " << oldestPage << "\n";
508                 }
509
510                 // Load the new page into memory
511                 memoryQueue.push(pageIndex);
512                 pageTable.insert(pageIndex);
513                 std::cout << "Page " << pageIndex << " loaded into memory.\n";
514             } else {
515                 // Page is already in memory
516                 std::cout << "Page " << pageIndex << " is already in memory.\n";
517             }
518         }
519
520         std::cout << "Process " << process.getId() << " execution completed.\n\n";
521     }
522
523 private:
524     int frameCount;
525     std::queue<int> memoryQueue;
526     std::unordered_set<int> pageTable;
527 };
528
529 int main() {
530     // Create a FIFO page replacement object with 3 frames
531     FIFOPageReplacement fifoPageReplacement(3);
532
533     // Create processes with page indices
534     Process process1(1, {0, 1, 2});
535     Process process2(2, {3, 4, 5});
536     Process process3(3, {0, 1, 2});
537     Process process4(4, {6, 7, 8});
538
539     // Load processes into memory using FIFO page replacement
540     fifoPageReplacement.loadProcess(process1);
541     fifoPageReplacement.loadProcess(process2);
542     fifoPageReplacement.loadProcess(process3);
543     fifoPageReplacement.loadProcess(process4);
544
545     return 0;
546 }

```

Thrashing

- Consider what occurs if a process does not have “enough” frames—that is, it does not have the minimum number of frames it needs to support pages in the working set. The process will quickly page-fault.
- At this point, it must replace some page. However, since all its pages are in active use, it must replace a page that will be needed again right away.
- Consequently, it quickly faults again, and again, and again, replacing pages that it must bring back in immediately.
- This high paging activity is called thrashing. A process is thrashing if it is spending more time paging than executing.
- This incident typically occurs when there are already so many processes running in a system.

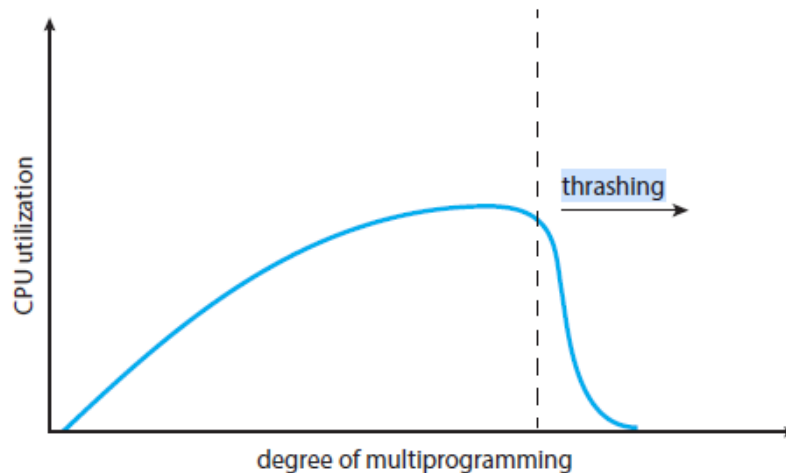


Figure 10.20 Thrashing.

Difference between paging and demand paging

- **Paging:**
 - Divides memory into fixed-sized blocks called pages.
 - When a process executes, it splits into pages.
 - When the process requests memory, the operating system allocates page frames from the primary memory.
 - The operating system moves program pages from the secondary memory (SSD/HDD) to the primary memory frames.
 - This technique improves the efficiency of memory management by eliminating external fragmentation.
- **Demand Paging:**
 - It's a specific implementation of paging.
 - The operating system loads only the necessary pages of a program into memory at runtime.
 - A page fault occurs when the program needs to access a page that is not currently in memory.
 - The operating system then loads the required pages from the disk into memory.
 - The operating system updates the page tables accordingly.
 - This process is transparent to the running program.

Memory-mapped files and shared libraries

- To further reduce the need for page replacement and maximize the efficient use of main memory – we can dedicate portions of the main memory for **shared data**.
- Summary: Processes should only have their dedicated memory space used for unique data. Any data that could be potentially shared must reside only once in the main memory.

Shared libraries:

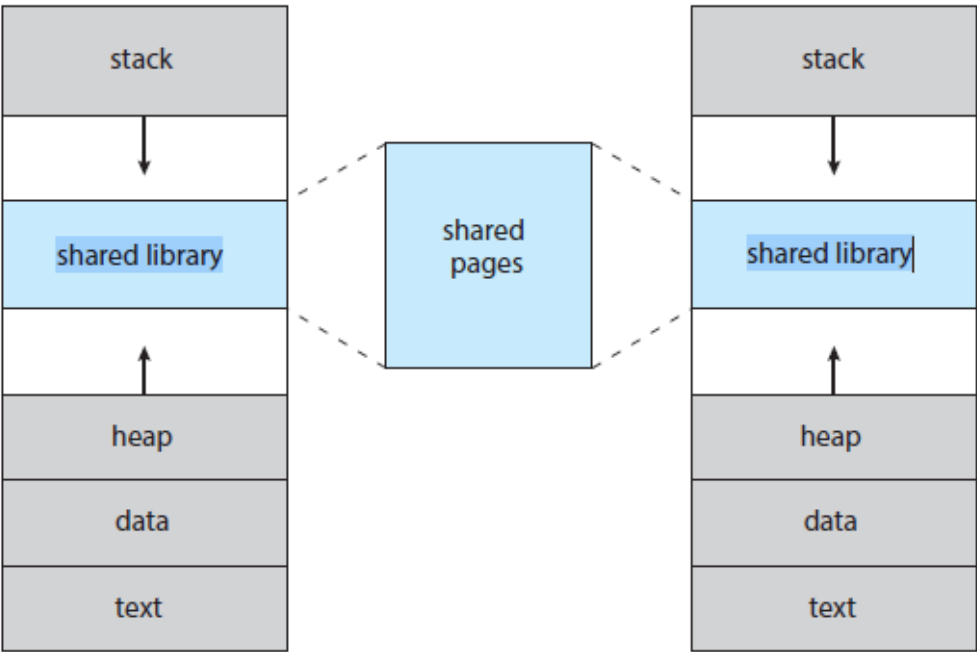
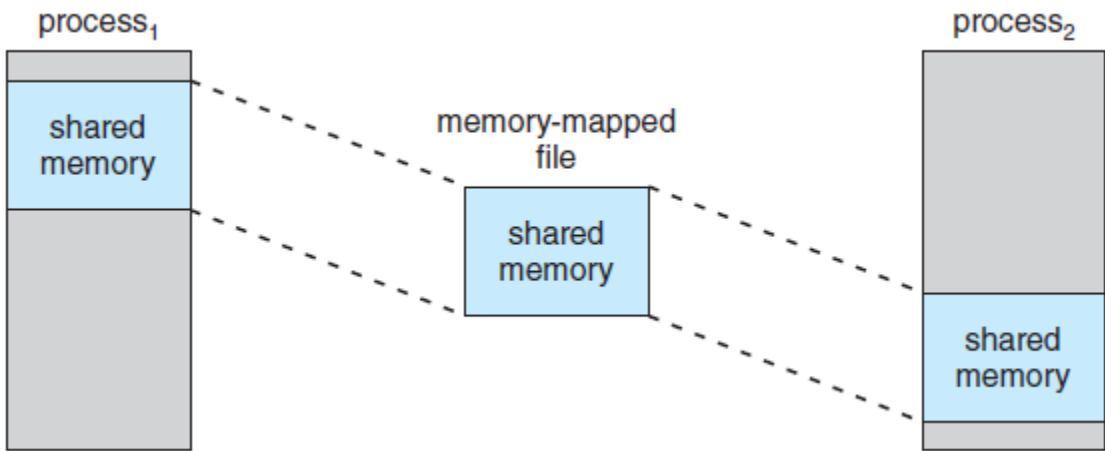


Figure 10.3 Shared library using virtual memory.

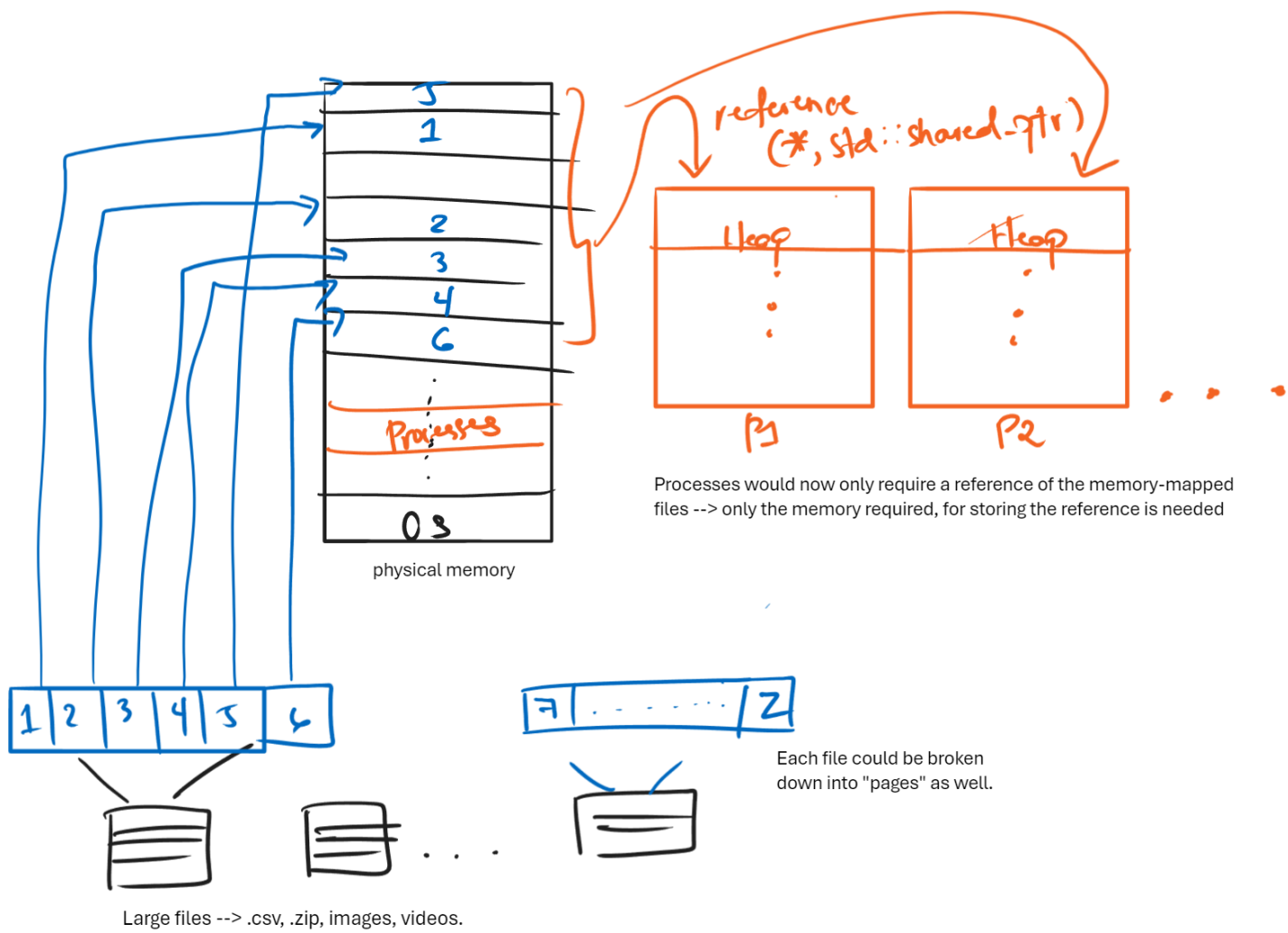
- System libraries such as the standard C library can be shared by several processes through mapping of the shared object into a virtual address space.
- Although each process considers the libraries to be part of its virtual address space, the actual pages where the libraries reside in physical memory are shared by all the processes (Figure 10.3). Typically, a library is mapped read-only into the space of each process that is linked with it.

Memory-mapped files:

- Recall that each process will have some pre-allocated memory.
 - Typically the heap/stack is pre-allocated and can run out of space, especially for operations that perform reading/writing of big datasets.
- Memory-mapped files are a mechanism that enables a file or a portion of a file to be directly mapped into the virtual memory of a process. This allows the process to access the file using memory references rather than traditional read and write operations.
- Memory-mapped files are commonly used in scenarios where a large dataset needs to be accessed in a way that is more efficient than traditional file I/O, such as database systems, certain types of file processing, and shared-memory IPC (Inter-Process Communication).



All programming languages provide support for memory-mapped files to allow developers to code applications that would require large files to be read.



BACKGROUND

- All processes would often require one or more resources. These resources are often limited → number of CPUs, amount of memory, storage available, I/O devices available.
 - At some point in time, there will be resource contention where multiple processes will attempt to get a certain resource.
- CPU cores will simply perform their usual pipelined instructions (fetch, decode, execute) → there must be an arbiter (the OS) that decides what set of processes can execute concurrently.
- In your OS emulator, there will be lots of processes that need to request the CPU, get memory, and update reports on CPU/memory utilization → all have possible contention/race conditions.
- There is a need to promote cooperativeness among processes. This is known as process synchronization.

Process synchronization is a fundamental concept in operating systems that coordinates and controls concurrent processes or threads to ensure orderly execution and prevent conflicts in shared resources. In a multitasking environment, where multiple processes or threads execute concurrently, it becomes crucial to synchronize their activities to maintain data consistency, prevent race conditions, and avoid conflicts that could lead to errors or system instability.

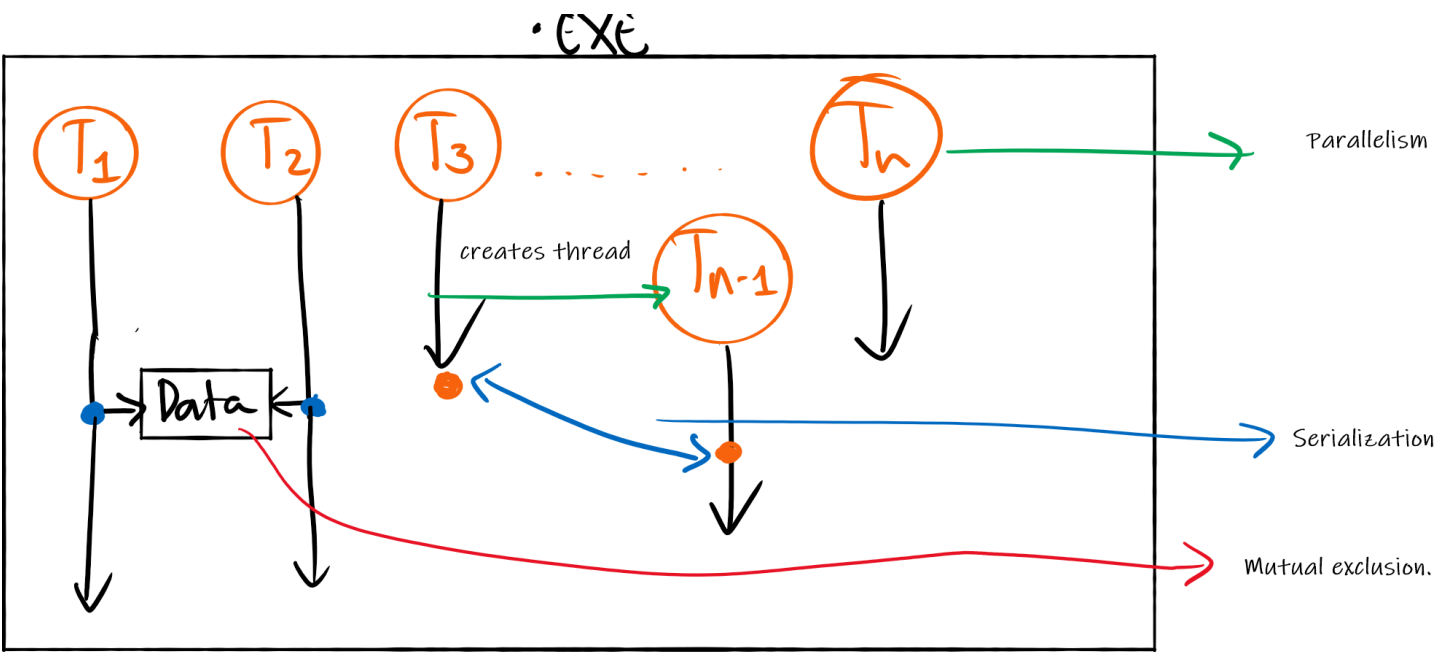
Motivation for Process Synchronization:

1. Shared Resources:
 - In a multi-process or multi-threaded environment, processes often share common resources such as memory, files, and devices. Proper synchronization is necessary to prevent conflicts and ensure that shared resources are accessed in a controlled manner.
2. Data Consistency:
 - Concurrent access to shared data structures can lead to inconsistencies if not synchronized. Process synchronization ensures that critical sections of code, where shared data is accessed or modified, are executed atomically.
3. Race Conditions:
 - Race conditions occur when the final outcome of a computation depends on the order of execution of concurrent processes or threads. Synchronization mechanisms help eliminate race conditions and establish a deterministic order of execution.
4. Deadlock Avoidance:
 - Processes waiting for resources may lead to deadlock situations where no progress can be made. Synchronization techniques, such as deadlock avoidance algorithms, prevent and resolve deadlock scenarios.
5. Preventing Starvation:
 - Fairness in resource allocation is essential to prevent situations where a process or thread is indefinitely delayed from accessing a resource. Synchronization mechanisms aim to ensure fairness and prevent starvation.
6. Orderly Communication:
 - Processes often communicate with each other through shared data structures or message passing. Synchronization facilitates orderly communication and prevents data corruption during inter-process communication.
7. Concurrency Control:
 - In database systems, synchronization is crucial for concurrency control to manage multiple transactions accessing the database simultaneously, ensuring consistency and isolation.
8. System Stability:
 - Uncontrolled access to shared resources or lack of synchronization can lead to unpredictable system behavior, crashes, or data corruption. Process synchronization contributes to overall system stability and reliability.

In summary, process synchronization is a cornerstone of operating systems, addressing the challenges that arise in concurrent computing environments. It ensures that multiple processes or threads can coexist and collaborate effectively without compromising data integrity, system stability, or overall performance.

Synchronization

- Attempting to perform multiple tasks at the same time in an organized manner.
- Developers who create concurrent systems are primarily concerned with:
 - Parallelism – Task A and B should execute at the same time to speed up performance.
 - Serialization – Task A should execute before Task B.
 - Mutual exclusion – Datatype X should only be modified by one task, A or B.



Why concurrency programming is difficult

Consider this sample program with two threads, Alice and Bob.

<pre>1 function Alice() { 2 eatBreakfast() 3 work() 4 eatLunch() 5 callBob() 6 }</pre>	<pre>8 function Bob() { 9 eatBreakfast() 10 eatLunch() 11 }</pre>
--	--

- First possibility – The computer is parallel. Alice and Bob **MAY** execute line 2 and 9 at the same time. In that case it is not easy to know if a statement on one processor is executed before a statement on another.
- Second possibility – Single processor is running multiple threads of execution. If there are multiple threads inside a single processor, then the processor can work on one for a while, then switch to another, and so on.

What is concurrency?

- The developer has no control over when each thread runs. The operating system makes those decisions (depending on the CPU scheduling scheme).
- The developer can't tell when statements in different threads will be executed.
- Two events are **concurrent** if we cannot tell by looking at the program/code which will happen first.

Introduction to Serialization

- Task A should execute before task B.
- callBob() passes a message to Bob, which is received by waitForCall(). As long as waitForCall() does not receive any message from callBob(), it cannot continue executing (thread sleeps).
- QUESTION: Where should we put waitForCall() to guarantee that Alice calls eatLunch() first?

<pre>1 function Alice() { 2 eatBreakfast() 3 work() 4 eatLunch() 5 callBob() 6 }</pre>	<pre>8 function Bob() { 9 eatBreakfast() 10 waitForCall() 11 eatLunch() 12 }</pre>
--	--

Adding waitForCall at line 10 puts the Bob thread to sleep, until callBob() is called by Alice thread. This is called **message passing**.

Alice thread execution – line A2 < A3 < A4 < A5. where **X < Y** means task X happens BEFORE task Y. **X > Y** means X happens AFTER task Y.

Bob thread execution – line B9 < B10 < B11

With message passing, it guarantees that: A4 < B11.

Activity:

Are the following statements true/false? Explain why.

- A2 < B9? – False. B9 can happen first.
- B9 < A3? – False. A3 can happen first.
- B11 > B10 > A5 > A4? – True. waitForCall() guarantees this.

Threads are non-deterministic!

- Concurrent programs are **non-deterministic**. It is impossible to tell by looking at the code, which will execute first.
- Non-determinism is one of the things that makes concurrent programs hard to debug. A program might work correctly 1000 times in a row, and then crash on the 1001st run, depending on the particular decisions of the scheduler.
- These kinds of bugs are almost impossible to find by testing; they can only be avoided by careful programming.

```
1 Thread_1() {
2     print("yes")
3 }
4
5 Thread_2() {
6     print("no")
7 }
```

Problem: Withdraw-Deposit

Consider this problem below.

<pre>int amount = 0; //deposit class Thread_A { void run() { amount = amount + 100; print(amount); } }</pre>	<pre>//withdraw class Thread_B { void run() { amount = amount - 100; print(amount); } }</pre>
--	---

```
1 int amount = 0;
2
3 //deposit
4 class Thread_A {
5
6     void run() {
7         amount = amount + 100;
8         print(amount);
9     }
10 }
```

```
12 //withdraw
13 class Thread_B {
14
15     void run() {
16         amount = amount - 100;
17         print(amount);
18     }
19 }
```

Critical section. Accessing this data can cause the following: 0, -100, +100

The three essential criteria for solving the critical section problem:

1. **Mutual Exclusion** - If process **P_i** is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the **n** processes

Peterson's Solution:

<pre>1 int amount = 0; 2 3 int turn = 0; 4 bool flag[2] = {false, false}; 5</pre>	<pre>7 //deposit 8 class Thread_A { 9 void run() { 10 flag[0] = true; 11 turn = 1; 12 13 while(flag[1] && turn == 1) { 14 //busy wait 15 } 16 17 amount = amount + 100; 18 print(amount); 19 20 flag[0] = false; 21 } 22 }</pre>	<pre>24 //withdraw 25 class Thread_B { 26 void run() { 27 flag[1] = true; 28 turn = 0; 29 30 while(flag[0] && turn == 0) { 31 //busy wait 32 } 33 34 amount = amount - 100; 35 print(amount); 36 37 flag[1] = false; 38 } 39 }</pre>
---	--	--

Explanation why it meets all three criteria:

- 1. Mutual exclusion – If Thread_A is in the critical section, then flag[0] is true. Thread_B will have turn = 0, which causes it to perform busy waiting. Vice versa, if Thread_B is in the critical section, then flag[1] is true. Thread_A will have turn = 1 which will cause Thread_A to perform busy waiting.
- 2. Progress – Once Thread_A finishes, it guarantees that Thread_B becomes the next to access the critical section because flag[0] becomes false, and vice versa.
- 3. Bounded waiting – Since flag[0]/flag[1] will become false in the future, this guarantees bounded waiting on the thread executing the while loop.

Activity:

Consider the pseudocode of Petearson's solution being used by two threads.

<pre>1 int amount = 0; 2 3 int turn = 0; 4 bool flag[2] = {false, false}; 5</pre>	<pre>7 //deposit 8 class Thread_A { 9 void run() { 10 flag[0] = true; 11 turn = 1; 12 13 while(turn == 1) { 14 //busy wait 15 } 16 17 amount = amount + 100; 18 print(amount); 19 20 flag[0] = false; 21 } 22 }</pre>	<pre>24 //withdraw 25 class Thread_B { 26 void run() { 27 flag[1] = true; 28 turn = 0; 29 30 while(turn == 0) { 31 //busy wait 32 } 33 34 amount = amount - 100; 35 print(amount); 36 37 flag[1] = false; 38 } 39 }</pre>
---	---	---

What do you think will happen from the above code?

Process synchronization for application-level programs (e.g. games, websites) is discussed on its own in a separate course. E.g. STDISCM /GDPARCM. We are going to discuss process synchronization on the OS level and how it's implemented. E.g. How do we represent processes? How do we create our own synchronization construct?

OPERATING SYSTEM CONTEXT

- The OS is designed to provide synchronization constructs that help developers ensure that their applications have their threads synchronized as well. E.g. use of mutexes, semaphores, monitors.
- What are the scenarios wherein process synchronization is required, at the OS level?
- The following cases commonly occurs in our day-to-day OS:

1. File Systems:

- In file systems, multiple processes or threads may attempt to access the same file concurrently. Synchronization mechanisms are implemented to control access to files, preventing conflicts and ensuring data consistency.
2. **Memory Management:**
 - Memory management involves allocation and deallocation of memory resources. Synchronization is crucial to manage access to shared memory spaces, preventing race conditions and ensuring that memory operations occur in a coordinated manner.
 3. **Device Drivers:**
 - Device drivers control the communication between the operating system and hardware devices. Synchronization is essential to coordinate access to device registers and buffers, ensuring that multiple processes or threads interact with devices in a controlled way.
 4. **Networking:**
 - Networking protocols and communication stacks involve concurrent processing of data packets. Synchronization is implemented to manage access to network buffers, preventing conflicts and ensuring the proper handling of incoming and outgoing data.
 5. **Process scheduling:**
 - The CPU cores and ready queues get updated for every CPU cycle. Thus, information about the availability of CPU cores and those in the ready queues must be synchronized to avoid re-scheduling already finished processes or possibly skipping processes in the queue.

Additional support for thread scheduling is provided for application developers:

- In multithreading environments, thread creation, termination, and synchronization require coordination. Mutexes, semaphores, and condition variables are employed to synchronize threads and manage shared resources.

TODO: Illustration of cooperative multitasking vs pre-emptive multitasking in 2000 operating systems.

Gist:

- Cooperative multitasking means that all processes must be programmed (by the developer) to yield control voluntarily
 - Problem: A process that spawns processes in an infinite while loop will hog the system and cause it to crash.
- Pre-emptive multitasking means that the operating system completely manages the scheduling, which decides when a process should be paused, and another process should be given CPU time.
 - This approach allows the operating system to preempt a running process to ensure fair allocation of CPU resources among all processes.
 - Pre-emptive multitasking improves system responsiveness and stability, as no single process can monopolize the CPU.
- Modern OS employs pre-emptive multitasking.

Consider a basic example of OS scheduling

Context:

- FCFS scheduler, with only one ready queue, but there are **4** cores that need to be utilized.
- The critical section is the ready queue, where individual schedulers constantly access the ready queue for getting processes to be executed.

```

1 //code to show an RR scheduler with mutex to access a single ready queue
2 #include <iostream>
3 #include <vector>
4 #include <thread>
5 #include <mutex>
6 #include <queue>
7 #include <chrono>
8 #include <random>
9
10 class Process {
11 public:
12     Process(int id, const std::string& name)
13         : id(id), name(name), numInstructions(10) {}
14
15     int getId() const {
16         return id;
17     }
18
19     std::string getName() const {
20         return name;
21     }
22
23     int getNumInstructions() const {
24         return numInstructions;
25     }
26
27 private:
28     int id;
29     std::string name;
30     int numInstructions;
31 };

```

```

33     std::mutex queueMutex;
34     std::mutex stopMutex;
35     std::queue<Process> readyQueue;
36     bool stopExecution = false;
37
38 void scheduler(int coreId) {
39     while (true) {
40         std::unique_lock<std::mutex> lock(queueMutex);
41
42         // Check if there are processes in the ready queue
43         if (!readyQueue.empty()) {
44             Process process = readyQueue.front();
45             readyQueue.pop();
46
47             lock.unlock();
48
49             // Simulate process execution
50             std::cout << "Core " << coreId << ": Executing Process " << process.getId()
51                 << " (" << process.getName() << ")" << std::endl;
52
53             int instructionsLeft = process.getNumInstructions();
54             while (instructionsLeft > 0) {
55                 std::this_thread::sleep_for(std::chrono::milliseconds(200)); // Simulating instruction execution time
56                 --instructionsLeft;
57             }
58         } else {
59             // Check if stopExecution flag is set
60             lock.unlock();
61             std::unique_lock<std::mutex> stopLock(stopMutex);
62             if (stopExecution) {
63                 // If stopExecution is true, exit the thread
64                 break;
65             }
66         }
67     }
68 }
69
70 void addProcessToQueue(const Process& process) {
71     std::lock_guard<std::mutex> lock(queueMutex);
72     readyQueue.push(process);
73     std::cout << "Process " << process.getId() << " (" << process.getName() << ") added to the ready queue." << std::endl;
74 }
75
76 int main() {
77     // Number of CPU cores
78     const int numCores = 4;
79
80     // Create CPU core threads
81     std::vector<std::thread> coreThreads;
82     for (int i = 0; i < numCores; ++i) {
83         coreThreads.emplace_back(scheduler, i + 1);
84     }
85
86     // Generate random processes and add them to the ready queue
87     for (int i = 1; i <= 10; ++i) {
88         Process process(i, "Process" + std::to_string(i));
89         addProcessToQueue(process);
90         std::this_thread::sleep_for(std::chrono::milliseconds(200)); // Simulating process arrival time
91     }
92
93     // Signal threads to stop execution
94     {
95         std::lock_guard<std::mutex> stopLock(stopMutex);
96         stopExecution = true;
97     }
98
99     // Wait for core threads to finish
100    for (auto& thread : coreThreads) {
101        thread.join();
102    }
103
104    return 0;
105 }

```

SYNCHRONIZATION CONSTRUCTS

- Synchronization constructs or data structures are made available to application developers.
- OS developers implement the behavior and validate the correctness of these constructs → atomicity, mutual exclusion guarantee, no race condition on sync constructs themselves.

For CSOPESY, we will only cover one construct, the **semaphores**. The most basic atomic construct would be the **lock**, which is simply a boolean flag of 1 (available), 0 (not available).

Semaphores

- Data structure that holds N number of permits. Two functions: **acquire(S)** where $S \leq N$. and **release(S)**.

Acquire(S)

- Attempts to obtain S permits. If insufficient permits are available, thread pauses and sleeps at this line.
- Otherwise, thread proceeds as is and acquires the permits.
- Note that this process is atomic. There is no race condition.

If insufficient permit, thread sleeps and pauses execution at this line. It will not continue.

```
1 Thread A() {
2   semaphore.acquire(2)
3   temp = count
4   count = temp + 1
5   semaphore.release(2)
6 }
```

```
1 Thread_A() {
2   semaphore.acquire(2)
3   temp = count
4   count = temp + 1
5   semaphore.release(2)
6 }
```

Releases 2 permits. At this point, any threads "waiting" at the acquire execution will check if permits are sufficient. If satisfied, permits are acquired and the threads are allowed to continue.

Release(S)

- Releases S amount of permits. Allowing other threads asking for permits to continue execution.

Declaring a semaphore

Semaphore name or label

```
1 semaphore = (5,5)
2
3 Thread_A() {
4   semaphore.acquire(2)
5   temp = count
6   count = temp + 1
7   semaphore.release(2)
8 }
```

Initial number of permits available

Maximum number of permits

Sample Problem: Mutual Exclusion Using Semaphores

Only 1 semaphore is needed.

<pre>1 int count = 0; 2 3 Thread_A { 4 temp = count; 5 count = temp + 1; 6 } 7 8 Thread_B { 9 temp = count; 10 count = temp + 1; 11 }</pre>	SOLUTION: <pre>1 int count = 0; 2 Semaphore mutex = (1,1); 3 4 Thread_A { 5 temp = count; 6 mutex.acquire(); 7 count = temp + 1; 8 mutex.release(); 9 } 10 11 Thread_B { 12 temp = count; 13 mutex.acquire(); 14 count = temp + 1; 15 mutex.release(); 16 }</pre>
--	--

Sample Problem: Multiplex

Using the solution in problem #1, allow K threads to enter at the critical section at any given point. Assume there are N threads running. $K \leq N$. For this problem, let **N = 10** and **K = 5**.

<pre>1 int count = 0; 2 3 Thread_A { 4 temp = count; 5 count = temp + 1; 6 } 7 8 Thread_B { 9 temp = count; 10 count = temp + 1; 11 }</pre>	SOLUTION: <pre>1 int count = 0; 2 Semaphore multiplex = (5,5); 3 4 Thread_A { 5 temp = count; 6 multiplex.acquire(); 7 count = temp + 1; 8 multiplex.release(); 9 }</pre>
--	--

Sample Problem: Simple Thread Barrier

Synchronize the threads to satisfy the following relation: $a1 < b2$ AND $b1 < a2$. a2 should wait for b1 to execute and b2 should wait for a1 to execute. This problem is called a **thread barrier**, where both threads must meet at a common checkpoint before a2 and b2 executes.

<pre>1 Thread_A { 2 a1(); 3 a2(); 4 } 5 6 Thread_B { 7 b1(); 8 b2(); 9 }</pre>	<pre>1 Semaphore A = (0,1); 2 Semaphore B = (0,1); 3 4 Thread_A { 5 a1(); 6 A.acquire(); 7 B.release(); 8 a2(); 9 } 10 11 12 Thread_B { 13 b1(); 14 A.release(); 15 B.acquire(); 16 b2(); 17 }</pre>
--	--

Programming Discussion

Discuss semaphore implementation through C++ mutexes – GDPARCM_HO4 example.

Activities: Use semaphores to solve the following problems.

- 1. Consider the **Producer-Consumer** problem where Producer thread waits for an event. When an event is detected, it is added into a buffer of MAX SIZE of 10. Consumer thread retrieves the latest event from the buffer and performs a task related to it. Assume that only 1 producer and 1 consumer thread is active.

<pre>1 int BUFFER = 10; 3 ProducerThread { 4 while(true) { 5 event = waitForEvent(); 6 buffer.add(event); //put event on a queue 7 } 8 }</pre>	<pre>10 ConsumerThread { 11 while(true) { 12 event = buffer.get(event); //gets event on a queue 13 doSomething(event); 14 } 15 }</pre>
--	--

Synchronize both threads using semaphores such that **waitForEvent()** and **doSomething()** can happen concurrently. However, only 1 thread gets to access the buffer (line P6 and C12). Show the pseudocode. TIP: You will need to use two semaphores and use BUFFER_SIZE as permit count. (10 pts)

SOLUTION:

```
1 int BUFFER = 10;
2 Semaphore prodSem = (BUFFER, BUFFER);
3 Semaphore consSem = (0, BUFFER);
4
5 ProducerThread {
6     while(true) {
7         event = waitForEvent();
8
9         prodSem.acquire();
10        buffer.add(event); //put event on a queue
11        consSem.release();
12    }
13 }
14
15 ConsumerThread {
16     while(true) {
17         consSem.acquire();
18         event = buffer.get(event); //gets event on a queue
19         prodSem.release();
20
21         doSomething(event);
22     }
23 }
```

- 2. Show the pseudocode using semaphores if there will be **multiple** producer and consumer threads active. TIP: You will need to introduce another semaphore (number of permits you must determine). Hence, 3 semaphores are required. (10 pts)

NOTE: Using the above solution, this can still result in a race condition. Multiple producer threads acquiring the prodSem can access the same buffer. Must use mutex semaphore.

```

1  int BUFFER = 10;
2  Semaphore prodSem = (BUFFER, BUFFER);
3  Semaphore consSem = (0, BUFFER);
4  Semaphore mutex = (1, 1);
5
6  ProducerThread {
7      while(true) {
8          event = waitForEvent();
9
10         prodSem.acquire();
11         mutex.acquire();
12         buffer.add(event); //put event on a queue
13         mutex.release();
14         consSem.release();
15     }
16 }
17
18 ConsumerThread {
19     while(true) {
20         consSem.acquire();
21         mutex.acquire();
22         event = buffer.get(event); //gets event on a queue
23         mutex.release();
24         prodSem.release();
25
26         doSomething(event);
27     }
28 }

```

How is atomicity coded at the OS level?

1. Atomic Instructions:

- **Atomic Operations:** Modern processors often provide atomic instructions, such as Compare-and-Swap (CAS), that perform read-modify-write operations in a single, uninterruptible step.
- **Atomicity Guarantee:** These instructions ensure that the operation appears to execute instantaneously and without interference from other processes or threads.

2. Hardware Support:

- **Transactional Memory (TM):** Some processors provide hardware support for transactional memory, allowing a sequence of memory operations to be treated as a single atomic transaction. If any part of the transaction fails, the changes are rolled back.
- **Atomic Read-Modify-Write (RMW) Operations:** Hardware may support atomic RMW operations like atomic increment, decrement, and test-and-set.

3. Interrupt Disable:

- **Disable Interrupts:** In certain cases, an OS or kernel might temporarily disable interrupts to achieve atomicity. This prevents interruption by hardware interrupts during critical sections.
- **Caution:** Disabling interrupts can have significant implications for system responsiveness and should be used judiciously.

4. Memory Barriers:

- **Memory Barrier Instructions:** Memory barriers, or fence instructions, ensure that memory operations before the barrier are completed before those after the barrier. They help control the order of memory accesses, enhancing atomicity.

5. Transactional Execution (Hardware Transactional Memory - HTM):

- **HTM Support:** Some processors support hardware transactional memory, where a block of code can be executed atomically as a transaction. If conflicts are detected, the transaction is rolled back.

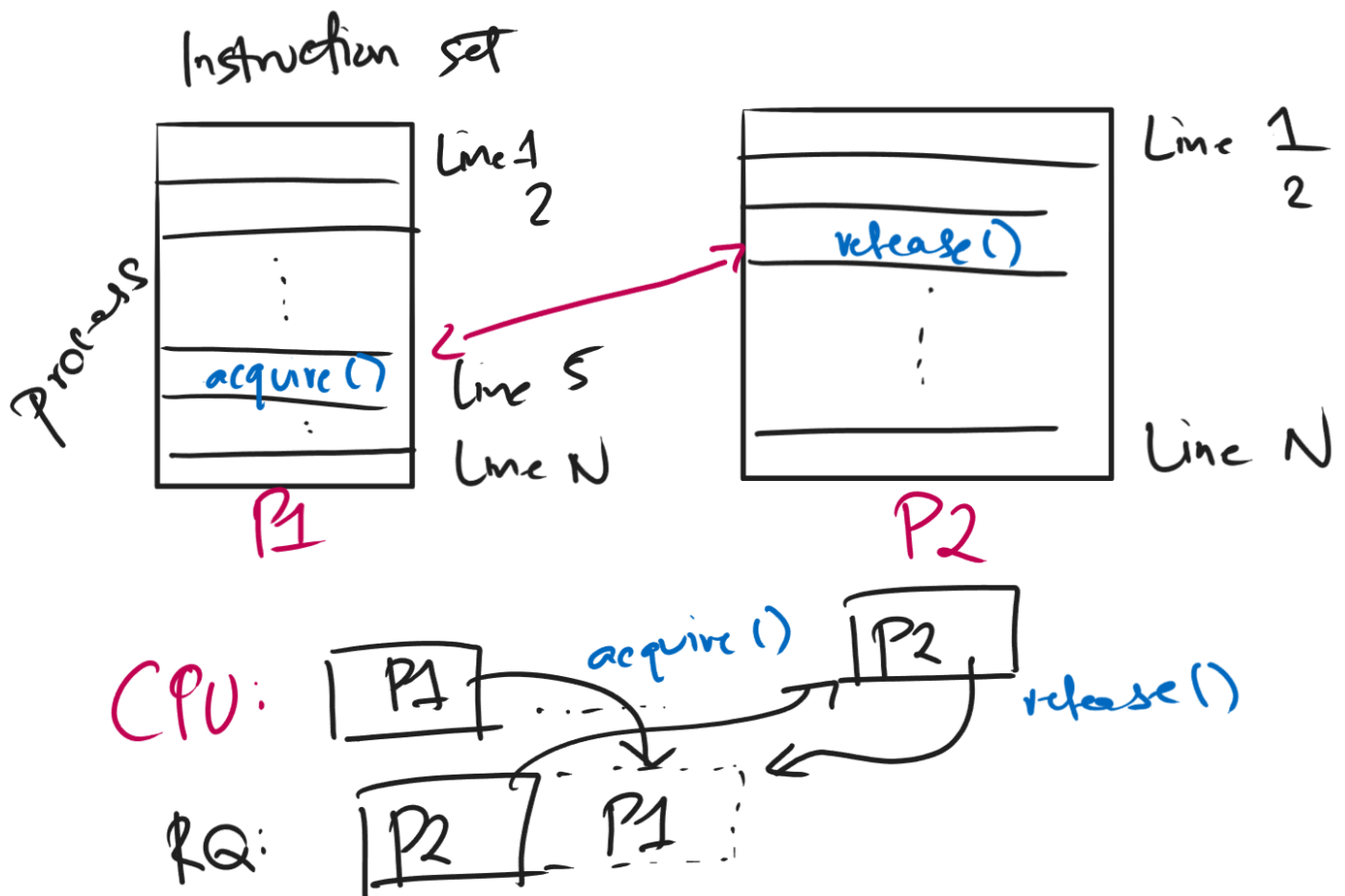
6. Atomicity in OS Kernels:

- **Critical Sections:** OS kernels use locks and other synchronization primitives to protect critical sections where shared data structures are accessed.
- **Preemption Control:** OS schedulers often include mechanisms to control preemption to avoid interference during critical operations.

Representing and emulating semaphores

What happens to a process if it encounters either acquire() or release() keywords in its instruction set?

Observation: A correct concurrent program would have its acquire and release system calls on different parts of a process/es. Consider the illustration below:

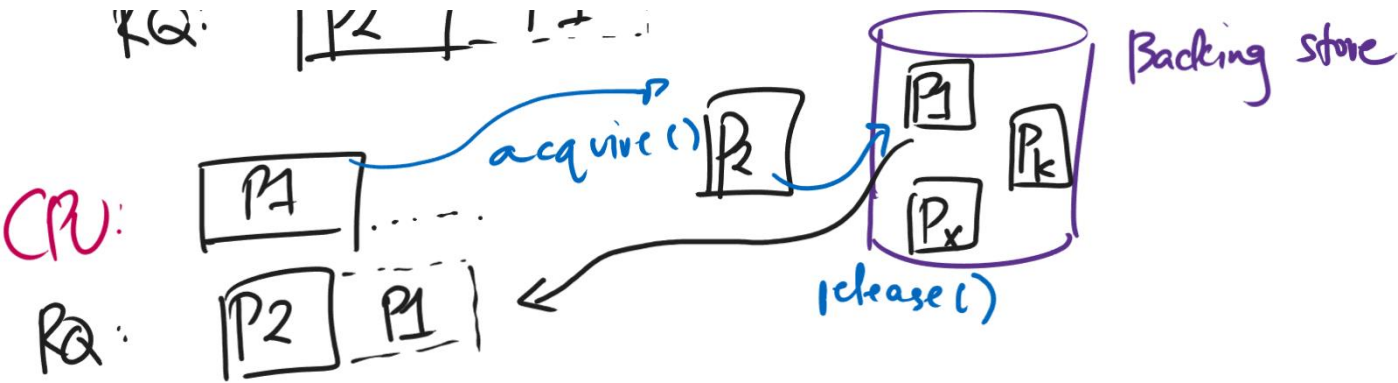


P1 is a sample process with an `acquire()` step. Thus, a correct synchronized program would have another process that would call `release()` → P2.

General steps (assume boolean mutex or a 0/1 semaphore):

1. When a CPU encountered an `acquire()` step during execution, it will immediately save the context of the process and put it back to the ready queue.
2. The CPU scheduler finds a new process to be executed.
3. If the sleeping process is currently selected from the ready queue, it will be skipped. E.g. `if(should_execute==false) → return back to RQ.`
4. One of the processes, containing the `release()`, would eventually be executed. If this happens, the processes which formerly called `acquire()` will become eligible for execution again.
5. That process, when selected again from the ready queue, will be executed. E.g. `if(should_execute) → put to CPU.`

Sometimes, an OS could be designed to hold a **sleep backing store**, independent from the ready queue, to provide a more distinguished view of currently sleeping processes. Thus, there would be no condition checking in the ready queue, since all processes inside it are guaranteed to be ready to execute.



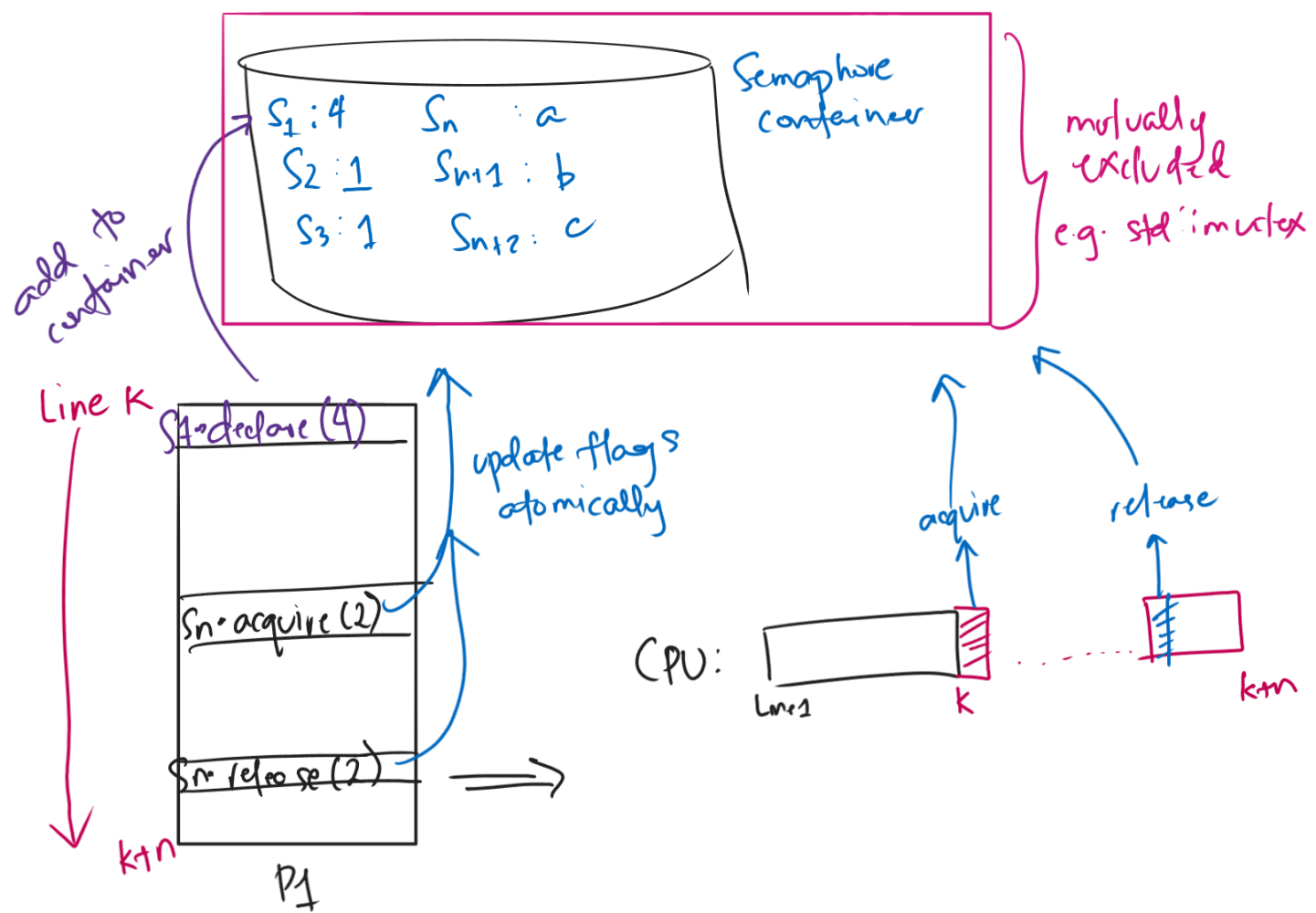
For counting semaphores, the general steps remain the same except a global counter for each declared semaphore is stored in memory.

Coding guidelines

To simulate semaphore behavior in your CSOPESY emulator, you must introduce a “**semaphore declare**” “**acquire**” and “**release**” command types in your process class. More details as follows:

Specific class (inherits ICommand)	Behavior
SemaphoreDeclare = "semaphore declare"	Declares a semaphore and store it in memory, such as a semaphore container.
Acquire	Proceeds to acquire flags from a given semaphore, referencing the semaphore container.
Release	Proceeds to release flags from a given container, referencing the semaphore container.

To ensure atomicity, the semaphore container must be mutually excluded, such as using mutexes before modifying the flag values.



Activity:

- Show your initial header files and provide placeholder implementations on how you would support semaphores in your CSOPESY emulator.
- Using the following test cases below, try emulating this behavior and check the expected output. The input format is as follows:


```

48 //first case
49 3
50 0 15 -has_semaphore=1 4 10 -num_flags=1 -acquire_pid=2 -acquire_how_many=1 -release_pid=2 -release_how_many=1
51 1 20 -has_semaphore=0
52 2 30 -has_semaphore=1 2 28 -num_flags=1 -acquire_pid=1 -acquire_how_many=1 -release_pid=1 -release_how_many=1
53
54 //second case
55 3
56 0 15 -has_semaphore=1 13 14 -num_flags=0 -acquire_pid=2 -acquire_how_many=1 -release_pid=2 -release_how_many=1
57 1 20 -has_semaphore=0
58 2 30 -has_semaphore=1 13 14 -num_flags=0 -acquire_pid=2 -acquire_how_many=1 -release_pid=2 -release_how_many=1
59

```
- The first line indicates how many processes must be declared.
- The input is a space-separated and variable-separated set of strings that follows the format: <PID> <num commands> <has semaphore> <acquire line> <release line> <num flags> <semaphore to acquire> <how many> <semaphore to release> <how many>
 - PID – the process ID $PID < N$, where N is the number of processes.
 - <num commands> - Number of commands in the process
 - <has semaphore> - 0/1. Indicates if a process has a semaphore. If 1, additional parameters are provided.
 - <acquire line> / <release line> - the line number/command index of where the acquire/release commands are. Command index starts at 0 and ends with (number of total commands – 1).
 - <num flags> - number of semaphore flags available
 - <acquire_pid> / <release_pid> - which process the semaphore must be acquired/released.

- <acquire_how_many> / <release_how_many> - how many acquires/release of flags must be performed once the acquire/release command is encountered.
- In the given two sample cases, the first should successfully execute both processes. When the user types “cpu-util”, the processes should be cleared from the CPU. The second case shows a deadlock. It should show the both processes indefinitely waiting for each other, indicated by an occupied ready-queue.